`codesign-mcdp`: A Python Library for Monotone Co-Design Problems

Version 0.1.0 – Reference Manual

May 2026

Abstract

`codesign-mcdp` is a Python library for formulating and solving *Monotone Co-Design Problems* (MCDPs) in the framework of Censi (2015). A design problem is a relation between two posets, a functionality poset F and a resource poset R; given a target functionality, the problem asks for the antichain of minimal resources needed to deliver it. Design problems compose under three operators (series, parallel, feedback), and the resulting class is closed under composition. The library implements the antichain calculus, six primitive design-problem types, the three composition operators, a Kleene fixed-point solver, and two high-level builders (an MCDPL-style declarative builder and a modular `System` builder). Further layers add set-based and stochastic uncertainty, compositional online learning, and temporal, vector-state, and online co-design, alongside a suite of worked examples. This document is the reference manual for version 0.1.0.

Contents

1 Introduction

A *co-design problem* is the joint design of components whose specifications mutually depend on each other. A canonical example: a battery must store enough energy to power an actuator, but the actuator must lift the battery itself, so the battery's mass appears in the energy it has to supply. There is no way to fix one without fixing the other.

Censi (2015) showed that under modest monotonicity assumptions, such problems have a clean algebraic theory: the relations between resources and functionalities form a category closed under series, parallel, and feedback composition, and solutions are computed by a Kleene fixed-point iteration in the lattice of antichains. The original publication included a software tool, `MCDPL`, distributed through `co-design.science`, which exposes the framework via a domain-specific language.

`codesign-mcdp` is a from-scratch Python implementation of the algorithmic core, designed around composable Python objects rather than a separate DSL. It targets users who want to formulate and solve MCDPs from inside their existing Python codebase, without context switching to a DSL or an external solver. The algorithmic core (posets, antichains, primitive design problems, composition, solver, and both builders) has no required runtime dependencies beyond the standard library; the uncertainty, online-learning, and visualisation layers pull in `numpy`, `scipy`, and `matplotlib` as optional extras. The whole package is roughly 8000 lines of Python and is licensed under MIT.

Scope of this document. This manual covers the public API and the mathematical structure it implements. Section 2 recalls the formal background. Sections 4 through 7 describe the data types, primitive design problems, composition operators, and the solver in detail. Section 8 covers set-based and stochastic uncertainty, and Section 9 the online-learning layer; Section 10 documents the visualisation helpers. Sections 11 and 12 document the two high-level builders. Section 13 walks through every example shipped with the library. Section 14 introduces the

temporal, vector-state, and online co-design layers, and Section 15 surveys realistic problem domains for them. Section 16 collects modelling guidelines and common pitfalls, and Section 17 discusses limitations and future directions.

Conventions. Code listings are typeset in monospaced font. Type signatures use Python's standard notation (`f: F → R`, for example). Mathematical quantities use F, R, h, $\top$, $\bot$, Min, etc.

2 Mathematical Background

This section recalls the formal definitions from Censi (2015) needed to specify the API. Proofs are omitted; the reader is referred to the paper for the full development.

2.1 Posets and Lattices

Definition 2.1 (Poset)**.** *A* poset *(partially ordered set) is a pair $(P, \leq)$ where $\leq$ is a reflexive, antisymmetric, transitive binary relation on the set P.*

Definition 2.2 (Bottom and top)**.** *An element $\bot \in P$ is a* bottom *(least element) if $\bot \leq x$ for every $x \in P$. An element $\top \in P$ is a* top *(greatest element) if $x \leq \top$ for every $x \in P$. A poset has at most one bottom and at most one top.*

Example 2.3 (Reals and Naturals)**.** *The non-negative reals augmented with infinity, $\overline{\mathbb{R}_{\geq 0}} = \mathbb{R}_{\geq 0} \cup \{+\infty\}$, ordered by the usual $\leq$, form a poset with $\bot = 0$ and $\top = +\infty$. Likewise $\overline{\mathbb{N}} = \mathbb{N} \cup \{+\infty\}$.*

Example 2.4 (Product order)**.** *Given posets $(P_1, \leq_1), \ldots, (P_n, \leq_n)$, the product $P_1 \times \cdots \times P_n$ with the componentwise order*

$$(x_1, \ldots, x_n) \leq (y_1, \ldots, y_n) \iff \forall i.\, x_i \leq_i y_i$$

is a poset. The bottom is $(\bot_1, \ldots, \bot_n)$ and the top is $(\top_1, \ldots, \top_n)$.

Definition 2.5 (Antichain)**.** *A subset $A \subseteq P$ is an* antichain *if no two distinct elements of A are comparable: $\forall x, y \in A,\ x \leq y \implies x = y$. We write $\mathsf{A}[P]$ for the set of all antichains of P.*

Definition 2.6 (Min operator)**.** *For any subset $S \subseteq P$, the* Min operator *returns the set of minimal elements:*

$$\mathsf{Min}(S) = \{x \in S : \nexists y \in S.\, y < x\}.$$

$\mathsf{Min}(S)$ is always an antichain. For a finite S, $\mathsf{Min}(S)$ is the Pareto front of S.

2.2 Antichains as a Lattice

The set of antichains of a poset carries a natural order: $A \leq A'$ in $\mathsf{A}[P]$ iff every point in A' is dominated by some point in A. Intuitively, $A \leq A'$ means A is "lower" or "easier", having weaker Pareto-minimal demands.

Proposition 2.7. *$(\mathsf{A}[P], \leq)$ is a complete lattice when P is finite or chain-complete. Joins are computed by union followed by Min. The bottom of $\mathsf{A}[P]$ is $\{\bot_P\}$ (or $\emptyset$ if P has no bottom); the top is $\emptyset$.*

This lattice is the state space of the Kleene iteration (Section 6.3).

2.3 Design Problems

Definition 2.8 (Design Problem)**.** *A* design problem (DP) *is a tuple $\langle F, R, h\rangle$ where F and R are posets and $h : F \to \mathsf{A}[R]$ is a function from functionality requests to antichains of resources. $h(f)$ is interpreted as the Pareto front of minimal resources required to deliver functionality f.*

Definition 2.9 (Monotone Design Problem)**.** *A DP $\langle F, R, h\rangle$ is* monotone *(an MCDP) if h is monotone with respect to the antichain order: $f \leq f'$ in F implies $h(f) \leq h(f')$ in $\mathsf{A}[R]$.*

Intuitively, monotonicity says that demanding more functionality cannot reduce the minimal resources needed. This corresponds to the common-sense engineering principle that you don't get more for less.

2.4 Composition

The class of MCDPs is closed under three operators.

Definition 2.10 (Series)**.** *Given DPs $d_1 = \langle F, M, h_1\rangle$ and $d_2 = \langle M, R, h_2\rangle$ whose interface poset M matches, their* series composition *is*

$$(d_1 \circ d_2)(f) = \mathsf{Min}\left(\bigcup_{r_1 \in h_1(f)} h_2(r_1)\right).$$

Definition 2.11 (Parallel)**.** *Given DPs $d_1 = \langle F_1, R_1, h_1\rangle$ and $d_2 = \langle F_2, R_2, h_2\rangle$, their* parallel composition *is $\langle F_1 \times F_2, R_1 \times R_2, h_1 \otimes h_2\rangle$ where*

$$(h_1 \otimes h_2)(f_1, f_2) = \mathsf{Min}(h_1(f_1) \times h_2(f_2)).$$

Definition 2.12 (Feedback)**.** *Given a DP $d = \langle F \times X, R \times X, h\rangle$ with a common factor X, the* feedback composition *on X is the DP $d^{\dagger X} = \langle F, R, h^{\dagger X}\rangle$ where $h^{\dagger X}(f)$ is the antichain obtained by closing the recursion on X.*

The recursion is solved by Kleene iteration in $\mathsf{A}[R \times X]$, seeded at $\{\bot\}$ and ascending until a fixed point is reached.

2.5 The Kleene Fixed-Point Algorithm

Theorem 2.13 (Censi 2015, Prop. 4)**.** *For an MCDP $d = \langle F \times X, R \times X, h\rangle$ closed on X, define the operator $\Phi_f : \mathsf{A}[R \times X] \to \mathsf{A}[R \times X]$ by*

$$\Phi_f(A) = \mathsf{Min}\left(\bigcup_{r \in A} h(f, r_X) \cap \uparrow r\right),$$

where r_X is the X-component of r and $\uparrow r$ is the upward closure. Then Φ_f is monotone, and the Kleene ascent

$$A_0 = \{\bot\}, \quad A_{k+1} = \Phi_f(A_k)$$

converges to the least fixed point, which equals $h^{\dagger X}(f)$ projected to $\mathsf{A}[R]$.

In implementation, the iteration terminates when one of three conditions holds: $A_{k+1} = A_k$ (fixed point), $A_{k+1} = \emptyset$ (infeasibility), or every point in A_{k+1} has $r_X = \top_X$ (loop axis saturates).

3 Installation and Project Layout

`codesign-mcdp` requires Python 3.9 or newer. The algorithmic core has no required runtime dependencies; the optional layers listed below pull in `numpy`, `scipy`, `matplotlib`, or `graphviz` as needed.

Installation. Clone the repository and install in editable mode:

```
git clone https://github.com/cbriat/codesign-mcdp.git
cd codesign-mcdp
pip install -e .
```

Optional extras.

- `pip install -e ".[viz]"` adds `matplotlib` for the visualisation helpers (Section 10) and the plotting examples.
- `pip install -e ".[online]"` adds `numpy` and `scipy` for the uncertainty (Section 8) and online-learning (Section 9) layers.
- `pip install -e ".[diagram]"` adds `graphviz` for the block-diagram renderer (Section 10.1).
- `pip install -e ".[nb]"` adds the Jupyter tooling needed to (re)generate the notebooks.

Repository layout.

```
codesign-mcdp/
  codesign/           the Python package (21 modules)
  examples/           23 runnable example scripts
  notebooks/          the same examples as executed .ipynb notebooks
  tests/              smoke tests
  docs/               this manual plus rendered trace images
  build_notebooks.py  script to regenerate the notebooks
  pyproject.toml      packaging metadata, PEP 621
  .github/workflows/  CI configuration
```

4 Core Data Types

This section documents the data types provided by `codesign.posets` and `codesign.antichains`.

4.1 Posets

The `Poset` abstract base class has the following interface:

```
class Poset(ABC):
    name: str

    def leq(self, x, y) → bool: ...      # is x ≤ y?
    def eq(self, x, y) → bool: ...
    def lt(self, x, y) → bool: ...
    def bottom(self): ...                 # least element, if any
    def top(self): ...                    # greatest element, if any
    def is_bottom(self, x) → bool: ...
    def is_top(self, x) → bool: ...
    def join(self, x, y): ...             # least upper bound
    def format(self, x) → str: ...
```

Four concrete poset classes are provided.

`Reals(unit="").` The non-negative reals augmented with $+\infty$. Bottom is 0.0; top is `math.inf`. The optional `unit` string is used for pretty-printing only.

`Naturals(unit="")`. The non-negative integers augmented with $+\infty$. Bottom is 0; top is `math.inf`.

`Ports(components: dict[str, Poset])`. The componentwise-ordered product of named factor posets. Elements are Python dicts mapping component names to values. The bottom is the dict of all bottoms; the top is the dict of all tops. `Ports` can be nested freely: a component poset may itself be a `Ports`. This is the type used for every design problem's F and R.

The name reflects the library's vocabulary: each named component is a *port* of the design problem, addressable by name in the constraint DSL. For backward compatibility the older name `NamedProduct` remains exported as an alias, so existing code that imports `NamedProduct` continues to work; new code should prefer `Ports`.

`Discrete(elements, leq=None)`. A finite poset with explicit element set and a caller-supplied `leq` predicate. Useful for catalog-like enumerations where the order is not the default discrete order.

4.2 Antichains

The `Antichain` class wraps a set of poset elements together with the poset they live in. The constructor normalises the input by removing dominated points and deduplicating.

```
class Antichain:
    poset: Poset
    points: tuple

    @classmethod
    def of_bottom(cls, poset) → "Antichain": ...
    @classmethod
    def empty(cls, poset) → "Antichain": ...
    @classmethod
    def singleton(cls, poset, point) → "Antichain": ...
    @classmethod
    def from_set(cls, poset, points) → "Antichain": ...

    def union_min(self, other) → "Antichain": ...
    def filter_above(self, floor) → "Antichain": ...
    def leq(self, other) → bool: ...
    def eq(self, other) → bool: ...
    def has_any_top(self) → bool: ...
    def is_empty(self) → bool: ...
```

The two key operations on antichains are `union_min`, which takes the union of two antichains and re-applies Min, and `filter_above`, which keeps only points dominating a given floor. Together they implement the body of the Kleene iteration in Theorem 2.13.

5 Primitive Design Problem Types

Six concrete subclasses of `DesignProblem` are provided, each appropriate for a different style of relation between functionality and resources.

5.1 `AlgebraicDP`

For relations where each resource is a closed-form monotone function of the functionality.

```
AlgebraicDP(
    F: Ports,
    R: Ports,
```

```
    equations: dict[str, Callable[[dict], float]],
    name: str = "algebraic",
)
```

The `equations` dictionary maps each resource name to a callable taking a functionality dict and returning a scalar. `h(f)` returns a singleton antichain.

Example 5.1. *A battery sized by specific energy:*

```
battery = AlgebraicDP(
    F=Ports({"capacity": Reals(unit="J")}),
    R=Ports({"mass": Reals(unit="kg")}),
    equations={"mass": lambda f: f["capacity"] / 1.8e6},
)
```

5.2 FunctionDP

For relations where the user supplies `h` directly as a function from functionality dicts to antichains. Use this when the relation is multi-valued (a true Pareto front) or has branching logic.

```
FunctionDP(
    F: Ports,
    R: Ports,
    h_fn: Callable[[dict], Antichain],
    name: str = "function",
)
```

Example 5.2 (A two-option amplifier)**.** *The same audio amplifier output spec admits two implementations: a class-A topology (clean, hot) or a class-D topology (efficient, harsh). `h(f)` emits both points, so the engineer sees the real tradeoff:*

```
F = Ports({"watts":    Reals(unit="W")})
R = Ports({"power_in": Reals(unit="W"),
           "thd":      Reals()})

def h_amp(f):
    watts = f["watts"]
    return Antichain.from_set(R, [
        {"power_in": watts / 0.25, "thd": 0.001},   # class-A
        {"power_in": watts / 0.90, "thd": 0.020},   # class-D
    ])

amp = FunctionDP(F, R, h_amp)
```

5.3 CatalogDP

For selecting from a finite catalog of implementations (motors, batteries, sensors).

```
CatalogDP(
    F: Ports,
    R: Ports,
    catalog: list[CatalogEntry],
    name: str = "catalog",
)

CatalogEntry(
    provides: dict[str, Any],
    costs: dict[str, Any],
    name: str = "",
)
```

For a request f, $h(f)$ is the antichain of `entry.costs` for every entry whose `provides` dominates f in F. Entries that are dominated by another feasible entry are pruned by Min in R.

Example 5.3 (A small motor catalog)**.** *Three motors with incomparable mass–cost tradeoffs at the same torque rating.* `h` *returns whichever entries cover the requested torque, then* Min *drops dominated ones automatically:*

```
motor = CatalogDP(
    F=Ports({"torque": Reals(unit="N*m")}),
    R=Ports({"mass":   Reals(unit="kg"),
                    "cost":   Reals(unit="USD")}),
    catalog=[
        CatalogEntry(name="Tiny",   provides={"torque": 2.0},
                     costs={"mass": 0.2, "cost": 30.0}),
        CatalogEntry(name="Light",  provides={"torque": 8.0},
                     costs={"mass": 0.5, "cost": 200.0}),
        CatalogEntry(name="Heavy",  provides={"torque": 8.0},
                     costs={"mass": 0.8, "cost": 120.0}),
    ],
)
# motor.h({"torque": 7.0}) yields the antichain
# Antichain[(mass=0.5, cost=200), (mass=0.8, cost=120)]
```

The Tiny entry is filtered out (does not cover 7 N·m). Light and Heavy are mutually incomparable, so both survive the Min.

5.4 ConstraintDP

A feasibility predicate plus a scalar cost, lifted to Min. Useful when the resource-functionality relation is most natural to express as *is this choice good enough, and how much does it cost?*.

```
ConstraintDP(
    F: Ports,
    R: Ports,
    sampler: Callable[[dict], Iterable[dict]],
    feasible: Callable[[dict, dict], bool],
    cost: Callable[[dict], dict],
    name: str = "constraint",
)
```

The `sampler` yields candidate implementations for the given functionality; `feasible(impl, f)` filters them; `cost(impl)` maps each feasible implementation into the resource poset, and the result is reduced by Min.

5.5 ODE_DP

Derives a monotone resource relation from a differential equation $\dot{x} = \mathrm{rhs}(x, t, f)$. Two modes are supported.

Final value integrates the ODE from $t = 0$ to $t = t_{\mathrm{end}}$ by explicit Euler and reports the extracted resources of the final state.

Steady state solves $\mathrm{rhs}(x, \cdot, f) = 0$ for x by Newton iteration and reports the extracted resources of the steady-state value.

```
ODE_DP(
    F: Ports,
    R: Ports,
```

```
    rhs: Callable[[state, float, dict], state],
    extract: Callable[[state], dict],
    mode: str = "final_value",   # or "steady_state"
    t_end: float = 10.0,
    n_steps: int = 200,
    x0_fn: Callable[[dict], state] = None,   # default: lambda f: 0.0
    name: str = "ode",
)
```

Example 5.4 (A heater at thermal steady state)**.** *A wall heater sized from Newton's law of cooling: to hold a temperature rise ΔT above ambient, the input power must balance the heat loss $k\,\Delta T$. Modelling the delivered power as a scalar state x that relaxes towards its steady value, $\dot{x} = k\,\Delta T - x$, the root is $x = k\,\Delta T$. The heater's resource is that steady input power:*

```
F = Ports({"delta_T": Reals(unit="K")})
R = Ports({"power":   Reals(unit="W")})
k = 0.5   # heat-loss coefficient, W/K

heater = ODE_DP(
    F, R,
    rhs=lambda x, t, f: k * f["delta_T"] - x,
    extract=lambda x: {"power": float(x)},
    mode="steady_state",
    x0_fn=lambda f: 0.0,
)
# solve(heater, {"delta_T": 30.0}) → Antichain[(power=15 W)]
```

The steady-state root $x = k\,\Delta T$ is recovered exactly by the Newton iteration. Note that the state carried by `ODE_DP` is a plain scalar (or a list of scalars), not a dict: `rhs`, `extract`, and `x0_fn` all operate on that state directly.

5.6 `UncertainDP`

Brackets an unknown h with a known lower and upper bound. The optimistic mode (`lower`) yields a lower bound on the minimal resources; the pessimistic mode (`upper`) yields an upper bound. Section VII of Censi (2015) develops the theory.

```
UncertainDP(
    F: Ports,
    R: Ports,
    lower: DesignProblem,
    upper: DesignProblem,
    mode: str = "upper",
    name: str = "uncertain",
)
```

`with_mode("lower" | "upper")` returns the bracket selected for solving.

Example 5.5 (A battery with uncertain specific energy)**.** *Specific energy of the cell chemistry is known only to lie in $[1.6, 2.0]$ MJ/kg. The optimistic and pessimistic brackets give lower and upper bounds on the required mass:*

```
F = Ports({"capacity": Reals(unit="J")})
R = Ports({"mass":     Reals(unit="kg")})

lo = AlgebraicDP(F, R, {"mass": lambda f: f["capacity"] / 2.0e6})
hi = AlgebraicDP(F, R, {"mass": lambda f: f["capacity"] / 1.6e6})

battery = UncertainDP(F, R, lower=lo, upper=hi)

# Solve both brackets and report the interval.
```

```
m_opt  = solve(battery.with_mode("lower"), {"capacity": 3.6e6})
m_pess = solve(battery.with_mode("upper"), {"capacity": 3.6e6})
# m_opt  → Antichain[(mass=1.80 kg)]
# m_pess → Antichain[(mass=2.25 kg)]
```

6 Composition Operators

The three operators of Section 2 are exposed as classes (`Series`, `Parallel`, `Loop`) with lowercase function aliases (`series`, `par`, `loop`) matching the paper's notation. Each takes existing design problems and returns a new `DesignProblem`, so composites nest freely and are solved by the same `solve` entry point. Because the class of MCDPs is closed under all three, a composite is itself a monotone design problem.

6.1 Series

```
series(dp1: DesignProblem, dp2: DesignProblem) → Series
Series(dp1, dp2)
```

Requires `dp1.R == dp2.F` structurally (same component names and posets). Implements the series definition: for each point of $h_1(f)$ run h_2 and take the union Min.

Example 6.1 (Battery feeding an actuator)**.** *A battery converts energy demand into mass; the actuator converts that mass demand into a power draw. Series composition feeds the battery's output directly into the actuator's input:*

```
battery = AlgebraicDP(
    F=Ports({"capacity": Reals(unit="J")}),
    R=Ports({"mass":     Reals(unit="kg")}),
    equations={"mass": lambda f: f["capacity"] / 1.8e6},
)
actuator = AlgebraicDP(
    F=Ports({"mass":     Reals(unit="kg")}),
    R=Ports({"power":    Reals(unit="W")}),
    equations={"power": lambda f: 10.0 * f["mass"] ** 2},
)
chain = series(battery, actuator)
# solve(chain, {"capacity": 3.6e6}) maps energy → power directly.
```

6.2 Parallel

```
par(dp1: DesignProblem, dp2: DesignProblem) → Parallel
Parallel(dp1, dp2)
```

Requires non-overlapping component names in F_1, F_2 and in R_1, R_2. The output F and R are the concatenated `Ports`s. The antichain is the Cartesian product followed by Min.

Example 6.2 (Two independent subsystems)**.** *A drone has a propulsion subsystem (energy in → mass out) and a sensing subsystem (sample rate in → power out). They share no ports and run in parallel; the composite DP takes both inputs and emits both outputs:*

```
propulsion = AlgebraicDP(
    F=Ports({"energy":      Reals(unit="J")}),
    R=Ports({"prop_mass":   Reals(unit="kg")}),
    equations={"prop_mass": lambda f: f["energy"] / 1.8e6},
)
sensor = AlgebraicDP(
    F=Ports({"sample_rate": Reals(unit="Hz")}),
```

```
    R=Ports({"sensor_pwr":  Reals(unit="W")}),
    equations={"sensor_pwr": lambda f: 0.05 * f["sample_rate"]},
)
combined = par(propulsion, sensor)
# combined.F has both energy and sample_rate;
# combined.R has both prop_mass and sensor_pwr.
```

6.3 Loop

```
loop(inner: DesignProblem, axis: str) → Loop
Loop(inner, axis="...")
```

Closes the feedback loop on a single named axis that must appear in both `inner.F` and `inner.R`. The resulting DP has `F = inner.F` minus the axis and `R = inner.R` minus the axis. Calling `h(f)` triggers the Kleene fixed-point iteration described in Theorem 2.13.

Example 6.3 (Battery mass feedback). *A drone's battery must carry both the payload and its own mass. Closing the loop on `battery_mass` converts the self-referential inequality into a fixed-point problem that the Kleene solver handles automatically:*

```
inner = AlgebraicDP(
    F=Ports({"payload":       Reals(unit="kg"),
             "battery_mass":  Reals(unit="kg")}),
    R=Ports({"battery_mass":  Reals(unit="kg")}),
    equations={
        "battery_mass":
            lambda f: 0.5 * (f["payload"] + f["battery_mass"]),
    },
)
drone = loop(inner, axis="battery_mass")
# drone.F = {"payload"}, drone.R = {}
# solve(drone, {"payload": 1.0}) iterates to the fixed point.
```

The closed-form solution is $m^\star = p/(1-0.5) = 2p$, the fixed point to which the Kleene iteration converges (creeping to machine precision over several dozen contraction steps).

Remark 6.4 (Multiple feedback loops). *`Loop` closes one axis at a time. To close several loops, chain them: `loop(loop(inner, axis="x"), axis="y")`. The `System` builder (Section 12) closes all feedback loops simultaneously over a single bundled axis, which is usually more convenient.*

7 The Solver

All design problems, primitive or composite, are solved through a single entry point, `solve`. It evaluates the problem at a given functionality, running the Kleene fixed-point iteration of Theorem 2.13 whenever a feedback loop is present, and returns a `SolveResult` carrying the resulting antichain together with convergence and feasibility metadata. This section documents that entry point, the underlying iteration, scalar minimisation over the returned front, and the solver's observability features.

7.1 Top-level entry point

```
solve(
    dp: DesignProblem,
    functionality: dict | None = None,
    max_iter: int = 200,
    *,
    trace: bool = False,
```

```
    verbose: int = 0,
    on_iteration: Callable[[TraceEntry], None] | None = None,
    start_from: SolveResult | Antichain | None = None,
    uncertainty: list[str] | None = None,
    n_samples: int = 1000,
    rng_seed: int | None = None,
) → SolveResult
```

The keyword-only arguments `trace`, `verbose`, `on_iteration`, and `start_from` are described in Section 7.4; `uncertainty`, `n_samples`, and `rng_seed` in Section 8. The result is a dataclass with the following fields:

```
@dataclass
class SolveResult:
    antichain: Antichain      # the Pareto front (in dp.R)
    iterations: int           # Kleene steps taken (0 if no loop)
    status: str               # "converged", "max_iter", or "diverged"
    feasible: bool            # True if antichain has finite, nonempty points
    trace: list[TraceEntry]   # the per-step trace if trace=True, else None

    # converged is a read-only alias for status == "converged"
```

Example 7.1 (Inspecting a solve result)**.** *The full lifecycle of a solve: build the DP, call* `solve`*, introspect the result, and pick a single design with* `minimize_cost`*:*

```
result = solve(drone, {"endurance":      300.0,
                       "extra_payload":  0.5,
                       "extra_power":    5.0},
               max_iter=200, trace=True)

print(result.status)          # "converged"
print(result.iterations)      # e.g. 17
print(result.feasible)        # True
print(result.antichain)       # Antichain[(total_mass=0.55 kg)]

# Scalarise: cheapest design under a custom cost function
best = minimize_cost(result, cost_fn=lambda r: r["total_mass"])
print(best)                   # {"total_mass": 0.5492}
```

7.2 The Kleene iteration

```
kleene_loop(
    loop_dp: Loop,
    f_outer: dict | None,
    max_iter: int = 200,
    *,
    trace: bool = False,
    verbose: int = 0,
    on_iteration: Callable[[TraceEntry], None] | None = None,
    start_from: Antichain | None = None,
    info_out: dict | None = None,
) → Antichain
```

The body of the iteration is direct from Theorem 2.13. The implementation adds:

- a *divergence cap* of 10^{30} that converts an iterate exceeding the cap to $\top$, so floating-point overflow becomes infeasibility rather than `inf`/`nan`;
- `try`/`except` around the inner h to catch `OverflowError`, `ValueError`, and `ZeroDivisionError`, again mapping to $\top$;

- three termination conditions: fixed point reached, antichain empty (infeasible), or every point's loop axis equal to $\top$ (provably infeasible).

7.3 Scalar minimisation over an antichain

```
minimize_cost(
    result: SolveResult,
    cost_fn: Callable[[dict], float],
) → dict | None
```

Returns the single resource bundle in `result.antichain` that minimises `cost_fn`, or `None` if the result is infeasible. This is the scalarisation step that turns a Pareto front into a single engineering choice.

7.4 Observability: trace, verbose, callback, status

The solver can be made to report its progress in three independent ways, and its termination reason is exposed through a structured `status` field that is orthogonal to `feasible`.

The `status` field. `SolveResult.status` is one of three strings:

`"converged"` The iteration reached a fixed point (or provably hit $\top$) within `max_iter` steps. The answer in `result.antichain` is the algorithm's verdict; `feasible` tells you whether that verdict is finite (a real design) or $\top$ (no design exists).

`"max_iter"` The iteration was cut off at `max_iter`. The current antichain may or may not be near a fixed point. Usually a sign to increase `max_iter`, or to look for an unintentional non-monotonicity in the model.

`"diverged"` At least one numeric component crossed the divergence cap of 10^{30} before the iteration could settle, and the solver stopped. Distinguishes pure numerical blow-up from clean $\top$-infeasibility.

The previous Boolean `converged` field is preserved as a backward-compatibility alias for `status == "converged"`.

Structured trace. Passing `trace=True` to `solve` populates `result.trace` with a list of `TraceEntry` dataclasses, one per Kleene step (plus the seed at iteration 0):

```
@dataclass
class TraceEntry:
    iteration: int           # 0 = seed, then 1, 2, ...
    antichain: Antichain     # snapshot at this step
    n_points:  int           # convenience: len(antichain)
    delta:     float | None  # max absolute change (numeric)
                             # or 0/1 (discrete); None at iter 0
    elapsed_ms: float        # wall time for this step alone
```

With `trace=False` (the default) the field is `None`, so the cost of unused tracing is zero.

Verbose printing. The integer `verbose` controls live output:

`verbose=0` (default) silent.

`verbose=1` one summary line at the end, e.g. `[solve] converged: 17 iters, |A|=1, total=2.0ms, feasible=True`.

`verbose=2` one line per iteration, showing the delta, the antichain size, and the step's wall time. Useful for watching oscillating fixed points.

Iteration callback. Passing `on_iteration=callable` registers a callback that receives each `TraceEntry` as soon as it is produced. Useful for live plots, custom logging, or interrupting an iteration that looks pathological:

```
def log_every_5(entry):
    if entry.iteration % 5 == 0:
        print(f"iter {entry.iteration}: delta={entry.delta:.3e}")

solve(dp, f, on_iteration=log_every_5)
```

Example 7.2 (Warm-starting a parameter sweep)**.** *When sweeping a parameter (load, mission length, demand level), the fixed point at one parameter is a great seed for the next. Pass the previous* `SolveResult` *(or its* `antichain`*) via* `start_from=` *and the Kleene iteration picks up from there instead of restarting at* $\bot$*:*

```
prev = None
results = []
for L in np.linspace(5.0, 30.0, 50):
    r = solve(dp, {"daily_load_kwh": float(L), ...},
              max_iter=400, start_from=prev)
    results.append(r)
    prev = r            # reuse the converged inner antichain
```

For the microgrid example (notebook 13) this reduces the cumulative iteration count by roughly 10 % across the sweep, more for finer parameter grids.

Example 7.3 (Plotting convergence after a trace)**.** *With* `trace=True`*, the result carries the entire iteration history; a single* `viz.plot_convergence` *call renders the delta-vs-iteration semilog:*

```
from codesign import viz

r = solve(drone, f, trace=True, max_iter=200)
ax = viz.plot_convergence(r)
ax.set_title(f"converged in {r.iterations} steps")
```

8 Uncertainty

Two kinds of parameter uncertainty are supported, declared on `Module` instances and consumed by the same `solve` entry point. Both are independent of the model's structure: a module can have set-based uncertainty, stochastic uncertainty, both, or neither.

8.1 Declaration

A `Module` carries optional attributes:

```
class Battery(Module):
    F = {"capacity": Reals(unit="J")}
    R = {"mass":     Reals(unit="kg")}

    def __init__(self, specific_energy=2.0e6, efficiency=0.9):
        self.specific_energy = specific_energy
        self.efficiency = efficiency
        super().__init__()

    def h(self, f):
        # nominal product 2.0e6 * 0.9 = 1.8 MJ/kg delivered,
        # matching the canonical drone (examples 1/6/7)
        return {"mass": f["capacity"]
                        / (self.specific_energy * self.efficiency)}
```

```
15
16 b = Battery()
17 b.uncertain_set  = Box(...)          # deterministic, set-based
18 b.uncertain_dist = Stochastic(...)   # statistical
```

The uncertainty solver discovers every such module by walking the `_codesign_modules` attribute that `System.build` attaches to the returned DP. Before each solve, the module's named parameters are reassigned; afterwards, their nominal values are restored. The user's original `Battery()` instance is unchanged.

8.2 Set-based uncertainty (deterministic)

`Box`. Axis-aligned interval product. Each parameter is declared as `(lo, hi)` or `(lo, hi, direction)`, where `direction` is one of the four tokens `"more_is_better"`, `"more_is_worse"`, `"less_is_better"`, `"less_is_worse"`. With all directions declared, the worst case is a single corner read off in constant time:

```
b.uncertain_set = Box(
    specific_energy=(1.7e6, 2.3e6, "more_is_better"),
    efficiency=(0.83, 0.97, "more_is_better"),
)
# worst case = (specific_energy=1.7e6, efficiency=0.83)
```

When some directions are undeclared, all 2^n endpoint combinations are evaluated and the worst is kept; cheap when n is modest.

`Ellipsoid`. An n-D ellipsoid $(p-c)^\top \Sigma^{-1}(p-c) \le 1$ in parameter space. The covariance Σ encodes both scale and correlation between parameters, so an ellipsoid is a more honest model than a box when parameters are believed to vary together:

```
b.uncertain_set = Ellipsoid(
    center={"specific_energy": 2.0e6, "efficiency": 0.9},
    cov=[
        [1.0e10, -2.0e3],
        [-2.0e3,  2.5e-3],
    ],
    params=["specific_energy", "efficiency"],
    directions={
        "specific_energy": "more_is_better",
        "efficiency":      "more_is_better",
    },
)
```

With all directions declared, the worst case is the boundary point $c + L\,u^\star$, where L is the Cholesky factor of Σ and $u^\star$ is the unit vector in the direction of badness. Otherwise, the boundary is sampled (controlled by `boundary_samples`) and the worst sample is returned.

2D conveniences. `Disk(center, radius, ...)` and `Circle(center, radius, ...)` reduce to the isotropic ellipsoid $\Sigma = r^2 I$. For monotone systems with declared directions, the two are equivalent: the worst case lies on the boundary either way.

8.3 Stochastic uncertainty

`Stochastic`. A joint distribution built from named scipy-stats frozen marginals plus a copula:

```
from scipy import stats

b.uncertain_dist = Stochastic(
    marginals={
```

```
        "specific_energy": stats.uniform(loc=1.7e6, scale=0.6e6),
        "efficiency":      stats.uniform(loc=0.83, scale=0.14),
    },
    copula=GaussianCopula(correlation=[[1.0, 0.4],
                                       [0.4, 1.0]]),
)
```

Sampling is by the standard copula–marginal trick: the copula generates correlated uniforms in $[0,1]^d$; the marginal `ppf`s map them to the named parameter axes.

Copulas. `Independence()` is the default (coordinatewise uniform). `GaussianCopula(correlation)` uses the Gaussian copula with the given correlation matrix; samples are drawn by Cholesky factorisation followed by Φ on each coordinate. Subclassing `Copula` requires only one method, `sample_uniform(n, d, rng)`, so other copulas (e.g. Clayton, Gumbel) drop in without further plumbing.

8.4 Querying

The same `solve` entry point accepts an `uncertainty` list and returns a richer object:

```
result = solve(
    dp, f,
    uncertainty=["worst_case", "mean", "p95", "cvar95", "samples"],
    n_samples=1000,
    rng_seed=42,
)

result.worst_case        # SolveResult at the worst point of the set
result.mean              # dict[r_port → mean across MC samples]
result.p95               # dict[r_port → 95th percentile]
result.cvar95            # dict[r_port → CVaR at 95% level]
result.samples           # list[Antichain], one per MC sample
result.feasibility_rate  # fraction of feasible MC samples
```

The allowed labels are:

`"worst_case"` The deterministic worst over every module's `uncertain_set`. One canonical solve at the worst-case parameter point.

`"mean"`, `"p95"`, `"cvar95"` Marginal statistics over Monte Carlo samples, drawn from every module's `uncertain_dist`. The CVaR (conditional value at risk) at the 95% level is the mean of the worst 5% of samples.

`"samples"` The raw antichain per MC sample, for the user to aggregate however they like.

For a single solve call you typically observe the ordering

$$\text{nominal} < \text{mean} < \text{p95} < \text{CVaR95} < \text{worst\_case},$$

which separates the four standard ways of reporting an answer "under uncertainty." Notebooks 11 and 12 visualise this for the example-7 drone.

Example 8.1 (Box worst-case of a drone battery)**.** *Specific energy and efficiency of the battery cell live in declared ranges, each with a direction of badness. The worst-case mass is the corner where both parameters take their worst endpoint:*

```
bat = Battery()  # the Module subclass from example~7
bat.uncertain_set = Box(
    specific_energy=(1.7e6, 2.3e6, "more_is_better"),
    efficiency=(0.83, 0.97, "more_is_better"),
```

```
)
# ... wire `bat` into the drone system ...

result = solve(drone, f, uncertainty=["worst_case"])
worst_mass = list(result.worst_case.antichain.points)[0]["total_mass"]
# worst_mass = 0.5668 kg, versus 0.5492 kg nominal
```

Example 8.2 (Monte Carlo with a Gaussian copula)**.** *The same battery, now with stochastic specific energy and efficiency, correlated at* $\rho = 0.4$*. One* `solve` *call returns all four summary statistics:*

```
from scipy import stats

bat.uncertain_dist = Stochastic(
    marginals={
        "specific_energy": stats.uniform(loc=1.7e6, scale=0.6e6),
        "efficiency":      stats.uniform(loc=0.83, scale=0.14),
    },
    copula=GaussianCopula(correlation=[[1.0, 0.4],
                                       [0.4, 1.0]]),
)

res = solve(drone, f,
            uncertainty=["mean", "p95", "cvar95", "samples"],
            n_samples=1000, rng_seed=42)
# res.mean["total_mass"]    → 0.5506 kg
# res.p95["total_mass"]     → 0.5632 kg
# res.cvar95["total_mass"]  → 0.5647 kg
# len(res.samples)          → 1000
```

9 Online Learning

When a co-design problem has many discrete candidates (catalog entries, robot types, component families) and each candidate's inner solve is non-trivial, the naive "evaluate every entry" strategy is wasteful. The `codesign.online` module implements the compositional online learner of Alharbi, Dahleh & Zardini (`arXiv:2604.22624`): maintain history-dependent bounds on each candidate's inner-solve output, evaluate the most promising one under an upper-confidence rule, then prune any candidate whose lower bound is already dominated by the incumbent.

9.1 Optimistic evaluators

The bound machinery is encapsulated in a small class hierarchy. `OptimisticEvaluator` is the abstract base; subclasses implement `bound(candidate)` returning a pair of dicts `(lower, upper)` mapping each numeric R component to its current lower and upper bound at the queried feature point. Tighter bounds prune more candidates; looser bounds are always safe.

Three concrete evaluators are provided:

Monotonicity. `MonotonicityEvaluator(features, r_components)` assumes the inner-solve output is component-wise monotone in the named features. Given an observation at feature point f_0 with antichain-min R_0, any candidate whose features dominate f_0 has resources $\geq R_0$ (lower bound); any candidate dominated by f_0 has resources $\leq R_0$ (upper bound). Aggressive when applicable. Only correct if the monotonicity assumption genuinely holds: choosing a derived feature (e.g. `cost_per_capacity = unit_cost / (speed * payload)`) is often what makes this safe in practice.

Lipschitz. `LipschitzEvaluator(features, r_components, L)` assumes $|h(c_1) - h(c_2)| \leq L \cdot \|features(c_1) - features(c_2)\|$ (Euclidean distance, per R component). Each observation tightens the bound by a cone of slope L. The constant can be a scalar (same for every R component) or a dict per component. Safe default: with a sensible L the bound never prunes a Pareto-optimal candidate.

Linear-parametric. `LinearParametricEvaluator(features, r_components, min_obs=3, prior_box=No noise_bound=0.0, solver="highs")` implements the *certified* optimistic bound of Alharbi *et al.* (Section V-C3, eqs. 26–28 and Lemma V.5). It assumes each resource coordinate is an *exact* affine function of the features, $\text{req}_k(c) = \phi(c)^\top \theta_k^*$ with $\phi(c) = [1, features(c)]$ (the paper's noiseless linear model). Rather than fit a single least-squares line, it maintains the whole *confidence polytope* $\Theta(H)$ — the subset of the prior box Θ_0 consistent with every observation taken as a linear equality — and returns, per resource coordinate, the coordinatewise minimum of the predicted resource over that polytope, $[\text{req}_{\text{opt}}(i, H)]_k = \min_{\theta \in \Theta(H)} \phi(i)^\top \theta$. Each such minimum is one linear program, solved with `scipy.optimize.linprog` (the `solver` kwarg selects the method). Because the true parameter θ_k^* is feasible for the LP, its optimum is always $\leq \phi(i)^\top \theta_k^* = \text{req}_k(i)$: the returned value is a *guaranteed* lower bound (the optimism guarantee of Lemma V.5), so — unlike the former OLS ± confidence-band heuristic — it can *never* wrongly eliminate a Pareto-optimal candidate. No upper bound is certified, so the returned upper bound stays at $+\infty$. `prior_box` sets Θ_0 (a per-parameter box) to keep the LP bounded while the fit is under-determined: `None` leaves every parameter unbounded, which is always safe but yields the trivial bound until enough observations pin θ down; `noise_bound` > 0 relaxes each observation equality into a two-sided band of that half-width, a documented extension for bounded observation noise. The old `confidence` kwarg is deprecated and ignored (passing it emits a `DeprecationWarning`). Net effect: this evaluator is both safe *and* effective when the affine assumption genuinely holds, and degrades *safely* — to the no-information bound, never to a wrong elimination — when it does not.

Gaussian process. `GaussianProcessEvaluator(features, r_components, length_scale=0.3, sigma_f=1.0, noise=1e-3, confidence=2.0, min_obs=3)` fits a zero-mean GP with an RBF kernel and bounds new queries by a confidence band on the predictive standard deviation. Implemented in pure numpy. The right tool when the response surface has feature interactions or local nonlinearity: the GP still produces informative (if uncertified) bounds there, whereas the certified linear-parametric evaluator correctly degrades to the no-information bound once the affine assumption fails. When the map *is* affine, prefer the linear-parametric evaluator — its bound is then both certified-safe and tight.

Subclassing. Custom evaluators only need to override `bound(candidate)`; the base class handles observation recording and the default fallback bound of $(0, +\infty)$.

9.2 The online solver

```
result = solve_online(
    candidate_fn,          # candidate_fn(candidate) → DP
    functionality,         # outer F vector
    *,
    candidates,            # list of feature dicts
    evaluator,             # OptimisticEvaluator instance
    budget=None,           # max inner solves; None = unbounded
    max_iter=200,          # forwarded to each inner solve
    verbose=0,
    warm_start=None,       # seed indices or n for farthest-point
    picker="lcb",          # "lcb", "ucb", "random", or callable
```

```
)
```

Algorithm (a simplified Algorithm 2 from Alharbi et al.):

1. Bound every remaining candidate via the evaluator.
2. Eliminate every candidate whose lower bound is already dominated by the current incumbent antichain.
3. Pick the most promising survivor by UCB on its lower-bound sum.
4. Run the inner solve at that candidate; observe the result in the evaluator; merge into the incumbent via `union_min`.
5. Repeat until the candidates are exhausted or the budget is hit.

The returned `OnlineResult` exposes:

`antichain` Min over the evaluated, surviving candidates.

`n_evaluated`, `n_eliminated`, `n_candidates` Bookkeeping for the elimination cascade.

`evaluated_ids`, `eliminated_ids`, `incumbent_ids` Lists of indices into the original candidate list.

`history` Per-iteration log: `{pick, antichain, remaining, evaluated, eliminated}`.

Example 9.1 (Fleet sizing across 200 robot types)**.** *A logistics service has a 200-entry catalog of candidate robot types, each described by four features. The Lipschitz evaluator caps how fast the inner-solve output can change with the features, so a single observation tightens the bound on every other candidate. With the right L the answer is exact:*

```
def candidate_fn(robot):
    capacity = robot["speed"] * robot["payload"]
    return AlgebraicDP(F, R, {
        "total_cost":   lambda f, cap=capacity, uc=robot["unit_cost"]:
                        (f["target_throughput"] / cap) * uc,
        "total_energy": lambda f, ek=robot["energy_per_km"]:
                        f["target_range"] * ek * 24.0,
    })

ev = LipschitzEvaluator(
    features=["speed", "payload", "unit_cost", "energy_per_km"],
    r_components=["total_cost", "total_energy"],
    L={"total_cost": 300.0, "total_energy": 30.0},
)

result = solve_online(
    candidate_fn, mission,
    candidates=robot_catalog,        # list of 200 feature dicts
    evaluator=ev,
)
# result.n_evaluated   = 192 / 200
# result.n_eliminated  = 8
# result.antichain     = 5 Pareto-optimal robot types
```

Swapping in `MonotonicityEvaluator(features=["cost_per_capacity", "energy_per_km"], ...)` *cuts the evaluation count to about 13 by exploiting the structural monotonicity of the derived feature.*

9.3 Warm-start, picker strategies, and the GP evaluator

The basic online solver has three small extensions that significantly expand the tuning space without complicating the core algorithm.

Warm-start. A new `warm_start` argument to `solve_online` pre-populates the evaluator with seed observations before the picker takes over. It accepts either a list of candidate indices (manually picked corner runs) or an integer n that triggers a greedy farthest-point heuristic over the feature space. The motivation is concrete: the `MonotonicityEvaluator`'s lower bounds only tighten for candidates whose features dominate every observation in the partial order. Without observations at the low-feature corner where the Pareto front typically lives, the picker explores indiscriminately and the evaluator remains uninformative throughout the budget. In example 16, evaluating the example with four corner warm-start runs lifts Monotonicity's Pareto recovery from zero to one of four classes; a hybrid with one of the other evaluators is the natural next step for cases where this is not enough.

Pluggable picker strategies. A new `picker` argument selects the candidate-scoring rule. Built-in options: `"lcb"` (the previous default, minimises the sum of lower-bound components, pure exploitation of the optimistic estimate); `"ucb"` (lower bound minus a $\kappa\cdot$ (upper − lower) exploration bonus, tunable via the tuple form `("ucb", {"kappa": 1.0})`); and `"random"` (uniform baseline for comparing the value of structural priors). Custom callables are accepted and receive `(lower, upper, r_components, **kwargs)`. The strategies that matter in practice are LCB for confident priors and UCB with $\kappa \approx 0.3$ to 0.5 when the prior is uncertain enough that pure exploitation gets stuck in a local region.

Gaussian-process evaluator. A new `GaussianProcessEvaluator` class uses a zero-mean GP with an RBF kernel, implemented in pure numpy. Hyperparameters are `length_scale` (default 0.3, suitable for features in $[0, 1]$), `sigma_f` (default 1.0, rescaled per-output by the empirical observation standard deviation), `noise` (default 10^{-3} jitter), `confidence` (default 2.0σ for roughly 95% coverage), and `min_obs` (default 3). The GP is more expressive than the linear-parametric evaluator and captures local nonlinearity that a global linear fit misses. Since the linear-parametric evaluator became the certified confidence-polytope bound the two occupy clearly separated niches. The certified `LinearParametricEvaluator` is the tool of choice when the resource map is genuinely affine in the features: on the linear fleet catalogue of example 14 it recovers all five Pareto types (where the former OLS-band version wrongly dropped one). But the example 16 bioprocess effect model is markedly nonlinear (temperature U-shapes, a power-law failure rate), and there the certified bound correctly refuses to over-claim — it degrades to the no-information bound and recovers zero of the four Pareto classes rather than eliminate a candidate it cannot rule out. The GP, willing to extrapolate a smooth surrogate, stays informative on that same grid (recovering one of the four classes). Rule of thumb: certified linear-parametric when you trust the affine assumption (safe *and* effective), GP when you do not.

10 Visualisation

The `codesign.viz` module exposes four helpers built on `matplotlib` and a small GraphViz emitter:

`plot_antichain(result, axes)` Scatter the antichain on the chosen 2D or 3D axes. Accepts a `SolveResult`, an `UncertaintyResult` (uses its worst case), or a bare `Antichain`. The 2D version optionally shades each point's dominated region for visual clarity.

`plot_convergence(result)` Semilog plot of the Kleene delta against iteration index. Accepts a `SolveResult` with a trace or a trace list directly. Useful for diagnosing oscillating fixed points and tuning `max_iter`.

`plot_uncertainty(unc_result, port, nominal=None)` Histogram of the Monte Carlo samples for the named R port, with the nominal, mean, p95, and CVaR95 marked as vertical lines.

`to_dot(dp, name="codesign")` GraphViz dot string for the system structure: modules as boxes, outer ports as labelled rectangles, constraint connections as edges. Render with `dot -Tpng` or paste into an online viewer.

All four accept an optional `ax=...` argument so they compose into larger figures. The module is imported as `from codesign import viz`.

10.1 Block diagrams of Systems

The `to_dot` function above emits a plain string with module-level wiring. For richer visualisation, the library ships a dedicated diagram renderer in `codesign.diagram` that produces Simulink-style block diagrams:

One box per subsystem with the F (functionality) ports listed on the left and the R (resource) ports on the right. Each port is individually addressable, so edges attach to specific ports rather than to the module as a whole.

Outer F and outer R as separate nodes on the diagram's left and right margins. The system inputs and outputs are visible, not buried inside the constraint list.

Port-level edges whenever the constraint was written in the operator-overloaded form (`module1.r_port >= module2.r_port * ...`). Constraints defined by callables that cannot be introspected are rendered with a dashed edge from a small "λ" marker, so they remain visible.

Cycle detection via Tarjan's strongly-connected-components algorithm on the module-level constraint graph. Edges within any cycle of size ≥ 2 are coloured amber, making the Kleene-iteration loop visible at a glance.

The renderer returns a `graphviz.Digraph` object, which can be displayed inline in a Jupyter notebook or written to SVG / PDF / PNG via `dot.render(filename, format="svg")`:

```
from codesign import draw_system

dot = system.draw_diagram()        # convenience method on System
# or equivalently:
dot = draw_system(system, rankdir="LR", highlight_cycles=True)

dot.render("bioprocess", format="svg", cleanup=True)
```

Optional dependency: install via `pip install codesign-mcdp[diagram]` (which pulls the `graphviz` Python package), and ensure the `dot` binary is on `PATH` (`apt-get install graphviz` on Debian / Ubuntu, `brew install graphviz` on macOS).

Example 10.1 (Bioprocess block diagram)**.** *The worked example 15 (`make_bioprocess`) builds a four- module System with a single outer F (target titer) and three outer R aggregations defined by callables. The diagram renderer surfaces this structure compactly:*

```
from codesign import draw_system
dp = make_bioprocess(cell_line=CHO_K1, glucose_setpoint_mm=8.0,
                     annual_demand_kg=100.0)
draw_system(dp, name="bioprocess").render("bioprocess",
                                          format="svg",
                                          cleanup=True)
```

The resulting diagram shows the four subsystems (`cell`, `feed`, `bior`, `media`), the outer functionality `target_titer` feeding `cell.target_titer`, the metabolic-load chain `cell.peak_vcd`, `feed.metabolic_factor` → `bior.peak_vcd` and `media.peak_vcd`, and a dashed λ marker feeding the three outer R aggregations.

Example 10.2 (Cyclic drone modular system)**.** *Example 7's modular drone has a battery–actuator feedback loop: the battery's mass adds to the actuator's lift demand, and the actuator's electrical power sets the battery's required capacity. `draw_system` detects the cycle automatically and colours the two edges between `battery` and `actuator` in amber, while the supporting edges (from outer F endurance and extra_power into the cycle, and from each module's mass to the outer R total_mass) remain in muted grey.*

Example 10.3 (Pareto front of a vehicle co-design)**.** *The vehicle from worked example 8 has two Pareto-incomparable designs at the Small-parcel mission. A single call paints them on a mass–cost plane and shades the dominated quadrants so the front is visible at a glance:*

```
from codesign import viz
import matplotlib.pyplot as plt

result = solve(vehicle, {"payload": 2.0, "mission_energy": 2.0e5})
viz.plot_antichain(result, axes=["total_mass", "total_cost"])
plt.show()
```

For three resources, pass three axis names to get a 3D scatter.

Example 10.4 (Uncertainty histogram with summaries)**.** *Visualising the Monte Carlo output of a stochastic solve: the histogram of total mass across MC samples, with the nominal, mean, p95, and CVaR95 each drawn as a vertical line:*

```
res = solve(drone, f,
            uncertainty=["mean", "p95", "cvar95", "samples"],
            n_samples=1000, rng_seed=42)
ax = viz.plot_uncertainty(res, port="total_mass",
                          nominal=0.5492, bins=30)
```

The function reads the requested summaries off the result and labels them automatically.

Example 10.5 (System graph in GraphViz dot)**.** *For documentation or for understanding someone else's model, the constraint graph of a built `System` can be dumped as dot:*

```
dot = viz.to_dot(microgrid, name="microgrid")
# Render externally:
# echo "<dot output>" | dot -Tpng > microgrid.png
```

Modules become boxes, outer F ports become source rectangles, outer R ports become sink rectangles, and constraint connections become directed edges labelled with the linking expression.

11 The MCDP Builder

The `MCDP` class in `codesign.mcdpl` provides an MCDPL-style declarative syntax for a single self-contained design problem.

```
with MCDP("name") as m:
    m.provides("functionality_name", unit="...")    # outer F port
    m.requires("resource_name",     unit="...")    # outer R port

    m.constraint("resource_name", lambda f: <expr>)   # closed-form
    # or
    m.rule(lambda f: Antichain.from_set(...))         # multi-valued

    m.loop_on("axis_name")    # optional, may be repeated

dp = m.build()
```

The builder emits an `AlgebraicDP` or `FunctionDP` internally, then wraps it in `Loop`s as requested by `loop_on`. Multiple `constraint` calls on the same target are joined (max). Mostly useful when the model has a small number of variables and reads naturally as a list of $\geq$ inequalities, mirroring the paper's `mcdp { ... }` blocks.

Example 11.1 (Drone as a single MCDP block)**.** *The Fig. 48 drone rewritten as a flat MCDP, with one closed loop on* `battery_mass`*:*

```
with MCDP("drone") as m:
    m.provides("endurance",     unit="s")
    m.provides("extra_payload", unit="kg")
    m.provides("extra_power",   unit="W")
    m.provides("battery_mass",  unit="kg")   # loop axis: in F and R
    m.requires("battery_mass",  unit="kg")
    m.requires("report_mass",   unit="kg")   # mirror for visibility

    def battery_mass_eq(f):
        return (f["extra_power"]
                + 10.0 * (9.81 * (f["battery_mass"]
                                  + f["extra_payload"])) ** 2
                ) * f["endurance"] / 1.8e6

    m.constraint("battery_mass", battery_mass_eq)
    m.constraint("report_mass",  battery_mass_eq)
    m.loop_on("battery_mass")

drone = m.build()
```

Calling `solve(drone, {"endurance": 300.0, "extra_payload": 0.5, "extra_power": 5.0})` *runs the same Kleene iteration as worked example 1 and returns the same* `report_mass = 0.049 kg` *fixed point. The loop axis* `battery_mass` *must be declared in both* `provides` *and* `requires`*, since that is the name* `loop_on` *closes; the extra* `report_mass` *resource mirrors it into the outer R so the converged value stays visible in the result, following the guideline in Section 16.*

12 The System Builder

For larger designs the `System` class in `codesign.system` provides modular composition with named subsystems and algebraic connection constraints. This is the recommended way to build non-trivial co-design models. Two equivalent surface syntaxes are available: the operator-overloaded form (recommended for new code) and the legacy lambda form (still supported).

12.1 Operator-overloaded syntax (recommended)

`provides`, `requires`, and `add` each return a port handle. Arithmetic operators on handles build expression trees lazily; the `>=` operator registers a constraint with the parent system.

```python
from codesign import Module, Reals, System, solve

class Battery(Module):
    F = {"capacity": Reals(unit="J")}
    R = {"mass":     Reals(unit="kg")}
    def h(self, f):
        return {"mass": f["capacity"] / 1.8e6}

class Actuator(Module):
    F = {"lift_force": Reals(unit="N")}
    R = {"power":      Reals(unit="W")}
    def h(self, f):
        return {"power": 10.0 * f["lift_force"] ** 2}

sys = System("drone")
endurance     = sys.provides("endurance",     unit="s")
extra_payload = sys.provides("extra_payload", unit="kg")
extra_power   = sys.provides("extra_power",   unit="W")
total_mass    = sys.requires("total_mass",    unit="kg")

battery  = sys.add("battery",  Battery())
actuator = sys.add("actuator", Actuator())

battery.capacity    ≥ (actuator.power + extra_power) * endurance
actuator.lift_force ≥ 9.81 * (battery.mass + extra_payload)
total_mass          ≥ battery.mass + extra_payload

drone = sys.build()
```

The four port kinds are:

- **Outer F** (from `provides`): can appear on the RHS of constraint expressions. Cannot be a constraint target.
- **Outer R** (from `requires`): can be a constraint target. Cannot appear in expression RHS.
- **Module F** (`handle.f_port`): can be a constraint target. Cannot appear in expression RHS (its value is determined by the constraints on it, not by the current iteration state).
- **Module R** (`handle.r_port`): can appear in expression RHS. Cannot be a constraint target (its value is determined by the module's own `h`, not externally).

Misusing a port (e.g. trying to put a module F port on the RHS, or constraining an outer F) raises immediately with a message naming the port and explaining the rule. F and R port names within a single subsystem must be disjoint.

The supported operators are `+`, `-`, `*`, `/`, `**`, unary `-`, along with the helper functions `codesign.sqrt`, `exp`, and `log` for expressions that need transcendental terms. For demands that cannot be expressed algebraically, the legacy lambda form remains available and can be mixed freely with the operator form (see below).

12.2 Legacy lambda syntax

```python
sys.constrain("battery.capacity",
              lambda x: (x["actuator.power"] + x["extra_power"]) * x["endurance"])
sys.constrain("actuator.lift_force",
              lambda x: 9.81 * (x["battery.mass"] + x["extra_payload"]))
sys.constrain("total_mass",
              lambda x: x["battery.mass"] + x["extra_payload"])
```

The argument `x` is a dict carrying outer F values under their bare names (`x["endurance"]`) and subsystem R ports under their dotted names (`x["battery.mass"]`). Both syntaxes compile to the same internal representation; the operator form is purely sugar.

12.3 Class-based modules

The `Module` base class lets a design problem be defined declaratively:

```
class Battery(Module):
    F = {"capacity": Reals(unit="J")}
    R = {"mass":     Reals(unit="kg")}

    def __init__(self, specific_energy=1.8e6):
        self.specific_energy = specific_energy
        super().__init__()

    def h(self, f):
        return {"mass": f["capacity"] / self.specific_energy}
```

A `Module` subclass declares its F and R ports as class-level dicts and overrides `h(self, f)`. The constructor wires everything into the underlying `DesignProblem` machinery. Parameterised modules are written by overriding `__init__` and calling `super().__init__()` after setting any instance attributes. `h` may return a dict (treated as a singleton antichain), a list of dicts (multi-valued antichain), or an `Antichain` directly.

Example 12.1 (A modular drone end-to-end)**.** *The drone from worked example 7, built with the operator-overloaded constraint syntax. Each line below a* `System` *declaration is a textbook inequality:*

```
sys = System("drone")
endurance     = sys.provides("endurance",     unit="s")
extra_payload = sys.provides("extra_payload", unit="kg")
extra_power   = sys.provides("extra_power",   unit="W")
total_mass    = sys.requires("total_mass",    unit="kg")

b = sys.add("battery",  Battery())
a = sys.add("actuator", Actuator())

b.capacity   ≥ (a.power + extra_power) * endurance
a.lift_force ≥ 9.81 * (b.mass + extra_payload)
total_mass   ≥ b.mass + extra_payload

drone = sys.build()
r = solve(drone, {"endurance": 300.0,
                  "extra_payload": 0.5,
                  "extra_power": 5.0})
# r.antichain → Antichain[(total_mass=0.55 kg)]
```

`build()` bundles every subsystem's R into a single Kleene axis and closes the loop automatically. The constraint graph can be exported with `viz.to_dot(drone)` *for documentation.*

12.4 Semantics

The `build` method emits a `Loop` whose axis bundles every subsystem's resource poset:

$$\texttt{\_\_modules\_\_} = \prod_{m \in \text{modules}} R_m.$$

The inner DP performs one Kleene step:

1. Read the current bundle estimate to populate `ctx`[`m.r`] for every subsystem R port.

2. For each subsystem m, evaluate the constraint demands on its F ports to obtain f_m, then call $m.h(f_m)$ to obtain an antichain.

3. Take the Cartesian product across subsystems; for each combination, evaluate the outer-R constraint demands to produce a candidate point.

4. Return the resulting antichain.

The outer Kleene iteration converges over all subsystem R ports simultaneously.

12.5 Validation

`build` performs the following checks:

- every subsystem F port has at least one `constrain` target;
- every outer R has at least one `constrain` target;
- every constraint target refers to a known module F port or a declared outer R;
- module names do not clash with the reserved axis name `__modules__` and contain no `..`

If any check fails, `build` raises `ValueError` with a message identifying the missing or invalid item.

13 Worked Examples

The package ships 23 runnable examples under `examples/` and the same 23 as executed notebooks under `notebooks/`. Each is referenced below by its filename. This section walks through examples 1–17; the temporal examples 18–23 are covered in Section 14.

13.1 Example 1: the drone (Fig. 48 of the paper)

`01_drone.py`. The canonical MCDP from the paper. A battery powers an actuator that must lift the battery plus a payload, with extra fixed payload and avionics power. The model is a single `FunctionDP` closed by `Loop` on `battery_mass`. The example sweeps five mission profiles, three feasible and two infeasible. Iteration counts range from 8 to 41 depending on how close to the feasibility boundary the case is.

13.2 Example 2: integer optimisation (Sec. VI-D)

`02_integer_optimization.py`. The problem

$$x + y \geq \lceil \sqrt{x} \rceil + \lceil \sqrt{y} \rceil + c$$

over $\mathbb{N} \times \mathbb{N}$, parametrised by c. The inner DP enumerates all valid splits of the deficit, producing a multi-point antichain at each iteration. For $c = 4$ the iteration converges in six steps to $M(4) = \{(0,7),(3,6),(4,4),(6,3),(7,0)\}$. The example notes that the paper's claim $M(1) = \{(1,0),(0,1)\}$ contains a typo; the correct answer is $\{(0,3),(3,0)\}$, which is what the solver finds.

13.3 Example 3: AUV seabed surveying (Sec. VIII)

`03_auv_seabed.py`. An autonomous underwater vehicle sweeps an area A at velocity v with sensor radius r. Cyclic constraints couple time, energy, and cost. The example enumerates a small set of (v, r) tradeoffs per iteration, producing a three-point Pareto front per scenario.

13.4 Example 4: `UncertainDP` and `ODE_DP`

`04_uncertain_and_ode.py`. Two demos of the advanced primitives. First: a battery whose specific energy is known only to lie in $[1.6, 2.0]$ MJ/kg; the lower and upper brackets give optimistic and pessimistic masses. Second: a heater whose steady-state input power is recovered from Newton's law of cooling via `ODE_DP`.

13.5 Example 5: visualising the Kleene ascent

`05_visualize_kleene.py`. Uses matplotlib to render the sequence $A_0, A_1, \ldots$ for the Sec. VI-D problem, reproducing the structure of Fig. 36 in the paper. PNGs for $c \in \{1, 4, 8\}$ are committed under `docs/images/`.

13.6 Example 6: the drone in MCDPL syntax

`06_drone_mcdpl_syntax.py`. Rebuilds the drone with the `MCDP` builder. The model reads almost identically to the `mcdp { ... }` block in Fig. 48 of the paper. Output is identical to example 1.

13.7 Example 7: the drone, modular

`07_drone_modular.py`. Rebuilds the drone with the `System` builder. Battery and actuator are defined as `Module` subclasses with their own F and R, then wired together with three `>=` constraints in the operator-overloaded syntax. Fixed-point values match example 1 exactly; iteration counts are roughly $2\times$ because the bundle now has two coupled axes (battery mass, actuator power) instead of one.

13.8 Example 8: a modular vehicle with Pareto tradeoffs

`08_vehicle_modular.py`. Three independent subsystems: a `CatalogDP` motor (seven Pareto-incomparable entries), a linear chassis, and a battery. Five `>=` constraints close the loop on (motor, chassis, battery) resources. Two of the three mission profiles yield two-point system Pareto fronts (different motor choices trading total mass against total cost); the third is infeasible because no motor in the catalog can supply enough torque once the chassis grows to support the battery.

13.9 Example 9: a robotic arm with non-trivial topology

`09_robotic_arm.py`. Five subsystems (shoulder joint, elbow joint, sensor, controller, battery) wired with eight constraints. The connection graph is genuinely non-trivial: the controller's power demand depends on both the sensor rate and the joint command rate, the shoulder must support the elbow joint's mass at the end of its arm (introducing a mechanical cycle through `elbow.mass`), and the battery is sized by the integrated power of every electrical subsystem. The Kleene iteration resolves the resulting cycles in four steps for all three test mission profiles. This example is the cleanest demonstration of where the operator-overloaded syntax pays off: each constraint line states one physical relationship, and the cyclic structure emerges from the constraint graph rather than from explicit loop machinery.

13.10 Example 10: watching the solver work

`10_solver_trace.py`. The drone of example 7 instrumented to show every observability feature of `solve`: silent mode, the end-of-solve summary line (`verbose=1`), the per-iteration progress feed (`verbose=2`), the structured trace (`trace=True`), and a custom `on_iteration` callback. The same script runs the drone with three different `max_iter` caps and a deliberately under-resourced mission, demonstrating that `status` takes the three distinct values `"converged"`, `"max_iter"`,

and `"diverged"` on these inputs. The notebook version (`10_solver_trace.ipynb`) adds a delta-vs-iteration plot showing the characteristic oscillating decay of the Kleene iteration to machine precision.

13.11 Example 11: set-based deterministic uncertainty

`11_uncertain_drone.py`. The drone's battery has two internal parameters (specific energy, efficiency) whose nominal product, 1.8 MJ/kg delivered, reduces to the canonical drone of examples 1/6/7: the nominal solve converges to the same 0.5492 kg. Both are declared with "more is better" directions. First a `Box` with independent ranges, then a tilted `Ellipsoid`. The worst-case mass under the box is +0.0176 kg above nominal; under the ellipsoid it is +0.0035 kg. The discrepancy is the cost of admitting the implausible corner where both parameters are simultaneously at their worst; the ellipsoid model rejects it explicitly.

13.12 Example 12: stochastic uncertainty with a Gaussian copula

`12_stochastic_drone.py`. The same drone with two uniform marginals tied by a Gaussian copula at correlation 0.4. A single `solve` call requests all five summaries: `worst_case`, `mean`, `p95`, `cvar95`, `samples`. Over a thousand Monte Carlo samples, the five values come out as nominal < mean < p95 < CVaR95 < worst_case, the canonical ordering. The notebook adds a matplotlib histogram of the MC distribution with vertical markers at each summary.

13.13 Example 13: the microgrid flagship

`13_microgrid.py`. An off-grid cabin must supply daily energy and peak load from four subsystems: solar PV, lithium battery (with a discrete chemistry choice over LFP, NMC, LCO, NaIon), diesel generator, and structural frame. The frame's mass depends on the total supported mass *including its own*, so the Kleene iteration does real work; iteration counts of 20–25 are typical at the test mission. The example exercises every package feature in turn: catalog choice, cyclic coupling, warm-started parameter sweep over a 50-point load range (the second pass through the sweep, reusing the previous fixed point, takes roughly 10 % fewer total Kleene steps), stochastic sun hours via `Stochastic`, and the full visualisation suite.

13.14 Example 14: online elimination over a robot catalog

`14_online_fleet.py`. A logistics service must hit a target throughput and range from a catalog of 200 candidate robot types, each described by four features (speed, payload, unit cost, energy per km). The exhaustive solve runs 200 inner solves; the online solver from `codesign.online` prunes by an optimistic-evaluator bound. The example compares three flavours:

- Lipschitz with $L = 300$ for cost and $L = 30$ for energy evaluates roughly 190 / 200 candidates but provably recovers all Pareto-optimal types.
- Monotonicity on a derived `cost_per_capacity = unit_cost / (speed * payload)` feature evaluates only 13 / 200 candidates and still recovers every Pareto point.
- Linear-parametric, now the certified confidence-polytope bound, evaluates 122 / 200 candidates and provably recovers all five Pareto points. The former OLS ± 3-sigma heuristic pruned much more aggressively (about 35 / 200) but wrongly dropped one Pareto point on this seed; the certified LP trades that aggressiveness for a guarantee that it can never eliminate an optimal candidate.

The notebook visualises the elimination cascade in feature space with stars marking the true Pareto front.

13.15 Example 15: monoclonal antibody fed-batch co-design

`15_bioprocess.py`. A biopharmaceutical company has to deliver a monoclonal antibody at a target titer (g/L) and annual demand (kg/year). Four subsystems are coupled cyclically: a CHO cell line characterised by an effective specific productivity and an oxygen uptake rate; a media `CatalogDP` over commercial CD formulations (HyClone-CD, EX-CELL-CD-CHO, Cellvento-CHO-220, BalanCD-HIP); a bioreactor `CatalogDP` spanning single-use 200 L through stainless-steel 25,000 L formats; and a feed strategy parameterised by glucose set-point with a U-shaped batch-failure penalty.

The cycle that the Kleene iteration resolves: higher titer demand forces higher peak cell density, which inflates the metabolic burden through the feed strategy, which inflates the cell density required to deliver the same titer, which in turn drives the bioreactor and media catalog lookups. The outer Pareto front in $(\text{COGS}, \text{footprint}, \text{CO}_2 \text{ per gram})$ contains a genuine two-point tradeoff between CHO-K1 (low licence fee, long batch, larger footprint per year) and CHO-MK (high licence fee, short batch, smaller footprint at the same annual output).

All parameters are calibrated to ranges reported in the bioprocessing literature: specific productivity per cell line (Reinhart *et al.* 2019 [10]), oxygen uptake rates and $k_L a$ correlations, bioreactor capex, and media pricing, with the metabolic constraints taken from Khattak *et al.* 2010 [8] and Lao & Toth 1997 [11]. The example is the first biotech-upstream application in the library and is intended both as a teaching example for the framework and as a starting point for discussions with industrial process-development groups.

13.16 Example 16: online DOE for the mAb fed-batch process

`16_online_doe.py`. Process development almost never sees a choice between fundamentally different cell lines; that decision is made years before commercialisation. The day-to-day question is where to operate within a tight 4D box of process parameters around a fixed line. This example fixes CHO-K1 and the 100 kg/year mission from example 15 and sweeps over a $5 \times 5 \times 5 \times 3 = 375$-point grid of operating conditions (temperature, pH, glucose target, feed start day). The closed-form effect model that maps these conditions to effective productivity is calibrated from the process-parameter literature: temperature effects from Yoon *et al.* [5] and Sou *et al.* [6], pH and dissolved-oxygen effects from Trummer *et al.* [7], and glucose / feed effects from Khattak *et al.* [8] and Gagnon *et al.* [9]. Each candidate stands in for a 10 to 14-day bioreactor run costing \$20,000 to \$100,000 in materials, labour, and analytics.

The example compares two non-online baselines (a 75-run factorial DOE at the pH = 7.1 slice, and a 40-run uniform random sample at fixed seed) against the three online evaluators from `codesign.online` at a budget of 40 inner solves. At this budget the tuned Lipschitz evaluator recovers 3 of the 4 distinct Pareto classes, matching the 75-run factorial DOE at 53% of the experimental cost. The certified LinearParametric evaluator, by contrast, recovers *zero* classes here — and correctly so: this effect model is markedly nonlinear (temperature U-shapes, a power-law failure rate), so no single affine parameter set fits the observations, the confidence polytope cannot support a non-trivial bound, and the evaluator falls back to the no-information value rather than risk eliminating a candidate it cannot certify as suboptimal. This is the safe-degradation property in action: on example 14's genuinely linear catalogue the same evaluator recovers every Pareto point, whereas here it declines to guess. The Monotonicity evaluator alone is likewise uninformative, because its lower bounds tighten only for candidates above every observation in the partial order, and the picker has no observation at the low-feature corner where the Pareto front lives. The example flags both effects explicitly and discusses warm-start strategies. The companion notebook visualises the pick trajectory in the (temperature, glucose) projection.

The take-home message generalises beyond bioprocess: the structural prior is doing the work, the budget is the cost, and the prior must match the problem. A global predictive prior

propagates information across the whole grid only when its structural assumption holds — the certified LinearParametric evaluator is exact and effective on an affine map (example 14) but deliberately silent on this nonlinear one, while a Gaussian process buys reach on smooth nonlinear surfaces at the price of an uncertified bound. Pure local bounds (Lipschitz with conservative L, Monotonicity) are always safe but need either denser observations or a warm-start to concentrate.

13.17 Example 17: full-vehicle co-design across ICE, hybrid, and EV

`17_car_codesign.py`. The largest example in the package and the first to model a system with three architecturally distinct variants in a single design study. A passenger car is decomposed into 18 to 24 MCDP modules per architecture (24 for ICE, 23 for hybrid, 18 for EV), each with its own F (functionality demands) and R (resource outputs). The three architectures are:

- **ICE.** Engine block (catalog), forced induction (catalog), fuel injection, exhaust aftertreatment, cooling (parametric, sized to engine heat rejection), lubrication, multi-speed transmission (catalog), driveshaft / differential, fuel tank, alternator, 12V battery, starter motor.
- **Hybrid.** Atkinson-cycle engine (catalog), motor / generator, small HV battery (1 to 5 kWh, NiMH or Li-NMC), planetary power-split, power electronics (IGBT or SiC), plus the ICE accessories at reduced sizing. No alternator (DC-DC from HV), no separate starter (integrated starter-generator).
- **EV.** Traction motor (catalog, PMSM or induction), large HV battery (40 to 100 kWh, LFP or NMC, 400V or 800V), power electronics, on-board AC charger plus DC fast-charge interface, single-speed reducer, battery thermal management (heat-pump or resistive).

All three share a common chassis core (body frame, front and rear suspension, front and rear brakes, steering, tires, wheels) and a common auxiliary core (HVAC, interior trim, safety systems, lighting and infotainment).

Two coupled cycles close inside the constraint graph and are resolved by the Kleene iteration. The first is the mass spiral: every load-bearing subsystem reads the design mass as an F input, and the macro aggregation sums every module's weight back into the total. Heavier vehicle leads to heavier components leads to heavier vehicle; the fixed point is the smallest mass at which adding one kilogram of payload requires less than one kilogram of additional subsystem mass to support it. The second is the energy-storage loop: fuel-tank size (ICE / hybrid) or battery capacity (EV / hybrid) depends on consumption $\times$ range, and the storage's own mass contributes to curb weight. For EVs this cycle is dominant because the pack is 20 to 35% of curb weight, so convergence takes 30 to 50 iterations versus 15 to 25 for ICE.

The example provides four representative missions (Urban Compact, Family Daily, Suburban Utility, Performance) and sweeps every architecture across all four. For the Family Daily mission (5 passengers, 700 km range, 9 s 0-100): the cheapest ICE is \$35,523 with 196 g/km CO_2; the cheapest HEV is \$40,272 with 91 g/km CO_2 and the best 10-year TCO at \$56,427; the cheapest EV is \$64,666 with 58 g/km CO_2 but requires a 100 kWh 800V pack and ends up at 2434 kg curb weight. A 7-passenger 800 km EV remains infeasible against the modelled 2024 pack-level energy density (160 to 180 Wh/kg), matching the real-world absence of such a product.

Calibration values cite Genta and Morello [12] for the chassis, the Bosch Automotive Handbook [13] for engines, Pulkrabek [14] and Heywood [15] for ICE thermodynamics, Hofmann [16] for hybrid topologies, Naunheimer et al. [17] for transmissions, the IEA Global EV Outlook [18] and Larminie and Lowry [21] for the electric powertrain and battery pricing, Nemry et al. [19] for emission factors, the EPA Automotive Trends Report [20] for fleet averages, and Mock [22] for mass-versus-mission calibration. Numbers should be understood as "within published ranges for production vehicles in the 2020 to 2024 generation," not as production-precision OEM data; the framework's contribution is the compositional analysis, not the specific point values.

The block diagrams `docs/diagrams/car_ice.svg`, `car_hev.svg`, `car_ev.svg` show the full module graphs. The mass-spiral cycle is rendered in amber by the `draw_system` renderer (cf. section on diagram rendering), which detects strongly-connected components in the constraint graph via Tarjan's algorithm; in all three architectures the cycle includes every load-bearing module plus the powertrain.

14 Temporal, Vector-State, and Online Co-Design

The examples up to 17 solve a co-design problem *once*: a fixed functionality is posed and the Kleene iteration returns the minimal resource antichain. Many engineering systems are not static. The best architecture changes as the environment changes; a resource is spent or worn across a sequence of stages; the design must be re-decided online as measurements arrive. This section documents the six layers the package adds on top of the ordinary `solve()` to handle these regimes, and the theory that makes them exact. All six reuse a common decision object, the `Architecture` (a named wrapper around a `System`), so they compose with the whole rest of the framework.

14.1 Case 1: architecture switching (`temporal`)

The simplest temporal problem is *switching*: a system whose best architecture changes because the environment changes, with a cost to switch. The environment is a sequence of epochs, each posing its own functionality and admitting a subset of candidate architectures. `solve_schedule` chooses one architecture per epoch to minimise the total of the per-epoch co-design costs plus the switching costs, by an exact Viterbi (shortest-path) pass over the epoch-by-architecture lattice. The switching cost may be a uniform scalar or a per-transition callable, and a hysteresis parameter lets a brief unfavourable epoch be ridden out on the incumbent rather than switched into and back out of. The per-epoch cost is a full `solve()`, so switching sits on top of static co-design rather than replacing it.

14.2 Case 2, scalar value: dynamic programming (`dynamic`)

When a resource is carried *between* stages, the problem becomes a dynamic program. `solve_dynamic` runs a finite-horizon backward Bellman recursion whose per-stage decision is which architecture to instantiate, whose per-stage cost is itself a co-design `solve()`, and whose carried scalar state (fuel, charge, cumulative wear) is advanced by a user `transition`. The value function is scalar: at each (stage, state) it is the single best cost-to-go. The state is discretised onto an explicit `StateGrid`; a transition that would leave the grid envelope is rejected *before* snapping, so an over-spent resource (a negative fuel level) is never silently rescued by the nearest in-range node. The solver returns a state-indexed `DynamicPolicy`, queryable in closed loop at off-nominal states, and `rollout` threads it forward from a concrete initial state.

14.3 Case 2, antichain value: sequential co-design (`sequential`)

The multi-objective generalisation replaces the scalar value with a Pareto antichain. Following the sequential co-design theory, fix an ordered commutative resource monoid $(R, \leq, \oplus, 0)$ and work in the value space of upper sets of R ordered by reverse inclusion (a larger feasible set is a lower resource threshold, hence "easier"). A sequential co-design problem over stages $0, \dots, N$ and a state poset $(X, \leq_X, \bot_X)$ assigns to each stage k a state-parametrised minimal-resource map $h_k : X \to \mathcal{U}R$ and a transition $\phi_k : X \times R \to X$. The value is the backward recursion $V_{N+1} \equiv R$ and

$$V_k(x) \;=\; \mathrm{Min} \bigcup_{r \in h_k(x)} \Big(\uparrow r \;\oplus\; V_{k+1}\big(\phi_k(x, r)\big) \Big),$$

the antichain-valued Bellman operator, implemented by `Antichain.union_min`. In the package the accumulated resource (the named cost axes on the antichain) is kept deliberately *distinct* from the carried state x: the transition reads any carried quantity off the full solved point, while only the cost axes accumulate on the front. The scalar `dynamic` layer is the width-one special case (a single cost axis gives a singleton antichain).

Three results are made operational. **(Q1) Monotone value.** If, for every stage, (H1) h_k is monotone in the carried state (more carried state makes the stage no easier) and (H2) ϕ_k is monotone in the state, then every V_k is monotone. (H1) is load-bearing and necessary. `check_monotonicity` verifies both on the state grid; it is orientation-aware, accepting the benign "consumable but monotone" orientation (a budget carried as remaining slack) and flagging only genuinely non-monotone (perishable or fatigue-as-state) stages, which have an interior optimum and fall outside the guarantee. **(Q2) Front = reachable frontier.** $\mathrm{Min}\, V_k(x)$ equals the antichain of minimal reachable cumulative resources over feasible choice sequences; there is no tail-pruning gap, and the front size is the width of the reachable set, which grows polynomially in the horizon for a summed objective on a fixed number of axes (`sum_combine`) and stays bounded for a renewable one (`join_combine`). **(Q3) Exact factorisation.** At a reset stage (a quiescent transition landing on $\bot_X$ regardless of input), `detect_resets` and `factorise_at_resets` split the horizon into a $\oplus$-product of independent sub-problems, the order-theoretic deterministic analogue of regeneration-point decomposition; the proof uses only distributivity, so it holds in every regime.

14.4 The general carried state: vectors (`state`, `vector_dp`)

Realistic problems carry structured state, not a lone scalar. The Formula 1 seasonal co-design of Neumann, Zardini, and colleagues [23] carries a state vector of two battery-wear levels plus a discrete regulatory-penalty flag; a self-reconfiguring robot carries the accumulated wear of each drive module plus a shared energy budget. The `state` module provides a `VectorStateGrid`: a product grid over named axes, each either a `ContinuousAxis` (a bucketed real interval, snapped and bounds-checked like the scalar grid) or a `DiscreteAxis` (a finite labelled set with a caller-supplied order, or mutually incomparable when no order is given). `solve_vector_sequential` runs the same antichain-valued Bellman recursion carrying the full state vector, with the transition returning an {axis : value} mapping snapped onto the product grid and out-of-bounds successors rejected on every axis. The monotonicity results carry over verbatim with the componentwise product order substituted for the scalar order, and `check_vector_monotonicity` verifies (H1)/(H2) over that order. A single-axis grid reproduces the scalar `sequential` result exactly, so the vector layer strictly generalises rather than parallels it.

A subtle correctness point governs the backward pass. The stage antichain must not be Pareto-reduced on the cost axes *before* the transition is applied: a point dominated on cost may be the only feasible choice from a constrained carried state (a higher-cost morphology that spares a worn module), and its consequence for the carried state is invisible in the cost projection. The implementation therefore enumerates all solved points and lets `union_min` prune only after the transition. This is what makes the reconfigurable-robot example (21) feasible.

14.5 Precompute-then-DP: the Formula 1 structure

There are two distinct ways to combine co-design with dynamic programming, and they should not be conflated. The `sequential` solver above re-solves the co-design at every (stage, state), which is necessary when the per-stage functionality depends on the carried state. The Formula 1 seasonal framework [23] uses the cheaper opposite order: the co-design layer is run *once* to produce track-dependent Pareto-optimal performance-versus-wear mappings, which are then frozen into a catalog of implementations that an outer finite-horizon dynamic program selects among. No co-design solve happens inside the Bellman sweep; the DP only indexes the precomputed cat-

alog. The package exposes this explicitly: `precompute_catalog` solves the architectures once at a fixed functionality and returns the tagged Pareto catalog (retaining points that look dominated at the single-stage level, since they may become optimal once aggregated in the DP, exactly the F1 paper's observation), and `dp_over_catalog` runs the antichain-valued DP selecting catalog indices. Precompute-then-DP agrees exactly with `solve_sequential` when the co-design is state-independent, and is preferred there for cost; when the co-design genuinely depends on the carried state, only the re-solving `sequential` form is correct.

14.6 Online feedback co-design (`online_codesign`)

The layers above are open-loop planners: the whole horizon is solved in advance. The closed-loop counterpart re-decides the design online as measurements arrive. `run_online_codesign` implements a receding-horizon, measurement-in-the-loop controller: at each step it (i) senses the measured state of the running process, (ii) reads the live requirement and environmental conditions, (iii) re-solves the co-design at those conditions over the admissible architectures (`resolve_at` picks the cheapest feasible point), (iv) applies the chosen configuration by stepping the *true* plant, and (v) logs a `ControlStep`. Because the plant is stepped with the true process rather than a nominal transition, any divergence between plan and execution is absorbed by the next sensing step, which is what distinguishes feedback from open-loop replay. This is the co-design instance of control co-design (CCD, [25]) in its nested form, here the myopic variant (re-solve a single static co-design each step); the model is assumed known, so measurements update the carried state and the conditions but not the co-design model itself. Two natural extensions are a receding-horizon lookahead that plans several steps ahead with the vector DP and commits only the first, and online learning of the model from measurements (coupling in the online-learning layer of section 9); both are left for future increments.

14.7 Temporal worked examples (18–23)

Example 18: metabolic architecture switching

`18_metabolic_switching.py`. The first temporal example (Case 1). An organism alternates between a glycolytic architecture (cheap, capped growth) on glucose and a gluconeogenic one (costlier, higher ceiling, a fixed glyoxylate-shunt overhead) on acetate, with a re-acclimation cost to switch, the diauxic-shift lag. A contested `mixed` epoch, flanked by acetate, is served by the cheaper glycolytic pathway under a low switch cost (four switches) but ridden out on the incumbent under a high one (two switches): identical environment, different switching economics, qualitatively different metabolic program.

Example 19: rover module activation

`19_rover_modules.py`. Case 2 with a scalar carried state. A planetary rover activates and deactivates modules (drive, science, comms, survival) across mission phases on a battery that depletes and recharges from solar. The same `DynamicPolicy` runs science-heavy from a full battery and steps down to lighter modes as reserve falls, and holds a light mode throughout from a depleted start; the framing follows standard spacecraft power-mode practice.

Example 20: antichain-valued sequential co-design

`20_sequential_codesign.py`. A multi-leg survey whose value at each stage is a whole Pareto front of cumulative (cost, CO_2) totals. The solver returns the exact reachable frontier of whole-mission plans (all rapid, cheapest and dirtiest, through all eco, cleanest and costliest, with interior mixes), and demonstrates the polynomial (linear here) front-width growth and the passing (H1)/(H2) monotonicity guard.

Example 21: reconfigurable robot with a state vector

`21_reconfigurable_robot.py`. The vector-state layer. A field robot reconfigures between tracked, wheeled, and hybrid morphologies across five legs, carrying a three-axis state vector: per-module wear for two independent drive modules plus a shared energy budget. The morphologies are cost-incomparable (tracked cheaper on energy, wheeled on operations), so the mission Pareto front is a five-point (energy, ops) staircase, each point a different morphology schedule that spreads wear across the modules so neither hits its limit and forces an expensive fallback. This is the structured multi-component state of the F1 seasonal problem, in a robotics setting.

Example 22: online feedback co-design

`22_online_feedback_codesign.py`. The closed-loop layer. A solar-powered sensor node re-solves its co-design each step against its measured battery charge and the live data-rate demand. A storm raises the demand mid-run and the node escalates to a high-rate configuration; as the measured charge falls the re-solve is gated (a hungry configuration becomes infeasible when charge is low) and the node falls back through nominal to low-power, then climbs back as solar recovers. The audit trail records every decision, condition, and outcome. No offline plan is followed; each step is solved against what actually happened.

Example 23: hierarchical co-design for a Formula 1 season

`23_formula1_season.py`. A faithful reproduction, in the framework's vocabulary, of the seasonal co-design of Neumann, Habermacher, Fieni, Cerofolini, Zardini, and Onder [23], and the canonical instance of the precompute-then-DP structure of section 14.5. The two layers are explicit. The *race-level co-design* (layer one) is a genuine MCDP solve: for each track, battery size, and discretised incoming battery age, a `CatalogDP` over energy-deployment strategies emits a Pareto front of `(race_time, wear)`, since deploying more electrical energy lowers race time but ages the battery faster, and an aged pack (less usable energy) cannot reach the low race times a fresh one can. `precompute_catalog` freezes that front. The *season-level dynamic program* (layer two) is a scalar-maximisation MDP carrying the state vector $(w_{b,1}, w_{b,2}, x_{\mathrm{ex}})$, the fractional wear of the two regulation-permitted battery units plus a flag recording whether a replacement penalty has been incurred. At each race the controls are which unit to run, which frozen implementation (deployment strategy) to select, and whether to install a fresh unit (a grid penalty of ten places for the first replacement, five for each later one, per the modelled FIA rules). Race time maps to a finishing position through an empirical time-gap model and a probabilistic grid-start correction scaled by the track's overtaking difficulty, and the finishing-position distribution is integrated against the FIA points table to give the race's expected championship points; the DP maximises the season total by backward induction. No co-design solve happens inside the season sweep, which is what distinguishes this from the re-solving `sequential` DP.

Two of the paper's findings are reproduced. First, the optimal policy accepts a *local penalty for a global gain*: it takes a battery replacement at one race, sacrificing points that day (a worse grid slot), to keep a fresh, low-wear pack available for aggressive deployment across the remaining races, raising the season total. Second, race order shifts the optimal policy: because the grid-penalty cost is track-dependent (a replacement hurts far more at a hard-to-overtake circuit than at a low-downforce one), reordering the calendar moves the optimal replacement onto a cheaper track, so the attainable total is near-invariant while the policy adapts, the temporal coupling the paper emphasises. The season DP is validated against exhaustive brute-force enumeration (single- and mixed-battery unit configurations) in `tests/test_formula1.py`; the DP optimum matches the brute-force optimum exactly.

The example and notebook 23 also regenerate the paper's three key figures *in the same format*, for visual comparison: the race-time-versus-wear Pareto fronts per battery size and

initial age (the paper's Fig. 1, with the fastest-implementation node A and a tradeoff node B highlighted), the grid-start position penalty $\mu_{\text{pos}} \pm \sigma_{\text{pos}}$ with its zero-crossing at the reference slot, saturation beyond roughly P12, and harder-track ordering (the paper's Fig. 2), and the finishing-position density $\varphi(\text{Pos}_{\text{end}})$ for two grid starts (the paper's Fig. 3). These reproduce the paper's figure *structure and qualitative behaviour* from the framework's own co-design solves and position model; they are deliberately not a numerical match, since the paper's fronts come from an optimal-control lap simulation and a battery-health model, and its position model is fitted from FIA race data, none of which is reproduced here. The example labels the figures as such. The sense in which the framework "does the same thing" is that it produces the same structured artefacts (Pareto fronts, penalty curves, finishing densities) feeding the same season-level dynamic program, not that it recovers the paper's specific numbers.

15 Realistic Problem Domains for Temporal Co-Design

The temporal, vector-state, and online layers of section 14 were built with concrete application domains in mind. This section surveys realistic problems, drawn from the recent literature, that map onto these layers, as candidates for future worked examples and as evidence that the abstractions are not invented in a vacuum. Each paragraph names the co-design structure (which layer applies) alongside the domain.

15.1 Self-reconfiguring and modular robots

Self-reconfiguring modular robots deliberately change their morphology, by rearranging the connectivity of their modules, to adapt to terrain, task, or damage [26]: a worm shape for a pipe, legs for rough ground, a wheel for flat terrain. This is directly the temporal switching and vector-state setting: the morphology is the architecture, the terrain or task sequence is the environment, and per-module wear plus shared energy is the carried state vector (example 21 is a first cut). Recent work makes the coupling sharper. Terrain-aware morphology search in dynamic environments [27] chooses a morphology per scene under an energy budget; multi-task space applications map task requirements to modular-unit morphology through an inverse-mapping design step [28]; and reconfigurable cobots such as the CONCERT platform and freeform strut-node systems (FreeSN, SMORES) expose a catalog of admissible configurations with real reconfiguration costs, the switching-cost term of `solve_schedule`. Floor-cleaning tiling robots (hTrihex, hTetro) reconfigure to cover obstacle-strewn areas at lower energy than fixed-shape coverage planning [29], an energy-versus- coverage Pareto per configuration.

15.2 Programmable self-assembly

Programmable self-assembly designs building blocks with tunable binding interactions so they assemble into a target structure. The design problem is multi-objective, trading assembly *yield*, assembly *speed*, and *economy* (number of distinct particle species), which is exactly a Pareto-front co-design. Hübl and Goodrich show that simultaneously optimising assembly time and yield through the binding-energy and concentration design space can speed assembly by orders of magnitude without lowering yield, and that economical, semi-addressable designs (reusing building blocks) can beat fully-addressable ones on all three axes [30, 31]. The temporal angle is the assembly *protocol*: a staged schedule of temperature or concentration set-points is a sequence of co-design decisions coupled by the carried state of intermediate-species concentrations, a natural fit for the vector-state sequential DP, with Markov-state-model protocol optimisation [32] as the dynamic backbone.

15.3 Multi-objective protein design

Protein design inherently trades competing objectives, stability, function, solubility, immunogenicity, and manufacturability, and Pareto-based multi-objective sampling produces candidate sets spanning these tradeoffs for informed downstream selection [33]. This is a co-design antichain over design objectives. Recent methods make the front explicit: ST-PARM aligns inference-time generation to a controllable stability-versus-function tradeoff; MosPro and NSGA-2 variants produce Pareto-optimal sequence sets; MAProt negotiates a structure-versus-function consensus across agents. The temporal and online angle is closed-loop directed evolution: frameworks such as EVOLVEpro couple a generative or predictive model with iterative wet-lab feedback, proposing a batch, measuring it, and updating, which is precisely the online feedback co-design loop of section 14.6 with the measured functionality replacing the modelled one (the model- learning extension of that loop).

15.4 Control co-design (CCD)

Control co-design, the simultaneous optimisation of a physical plant and its controller, is an established engineering discipline, described by Garcia-Sanz as a game changer [25], with two standard paradigms: nested (an outer plant loop wrapping an inner control solve) and simultaneous [24]. The nested paradigm is exactly the structure of the online co-design loop. Concrete instances that map onto the temporal and online layers include robust control co-design with a receding-horizon MPC inner loop [34]; plant-and-controller optimisation for energy systems with MPC [35]; control co-design of spacecraft trajectory and rocket-engine design in NASA's OpenMDAO/GMAT tooling; digital-twin control co-design of active suspensions via reinforcement learning; and adaptive MPC with online recursive-least-squares identification for load-frequency control under renewable disturbances, the archetype of the model-learning extension. These supply both the vocabulary (plant, controller, nested vs simultaneous) and the benchmark problems (active suspension, hybrid-electric vehicle, airborne wind energy) for positioning the online layer.

15.5 Summary of the mapping

Across these domains the same three structural questions recur, and the package answers each: which architecture to run as conditions change (switching, `temporal`); how a resource or wear vector carried across stages shapes the optimal schedule (vector-state DP, `vector_dp`); and how to re-decide the design online from measurements (feedback, `online_codesign`). The reconfigurable robot (21) and adaptive sensor node (22) are first worked examples in the robotics and control-co-design cells of this table; self-assembly and protein design are identified as high-value future examples whose multi-objective, staged-protocol, and closed-loop-evolution structure the existing layers already support.

16 Modelling Guidelines and Pitfalls

A few patterns that come up repeatedly when authoring MCDPs.

Expose loop variables you want to inspect. When using the operator API directly, `Loop` projects its axis out of the outer R. To inspect the converged loop value, include it in the inner R under a *different name*: for example, both `battery_mass` as the loop axis and `report_mass` mirrored for outer-R visibility. The `System` builder handles this transparently.

Cap physical maxima. When a design variable has a physical ceiling ($v_{\max}$, $r_{\max}$), make h return a $\top$-valued antichain whenever the iterate exceeds it. The Kleene ascent will then converge to infeasibility rather than oscillating or diverging.

Use `FunctionDP` or `CatalogDP` for breadth. `AlgebraicDP` always returns a singleton antichain. For a genuine Pareto front, use `FunctionDP` (enumerate the tradeoffs explicitly) or `CatalogDP` (provide multiple incomparable entries).

Scalarisation is downstream. The MCDP solver returns the Pareto front. Choosing which point to ship is a separate concern: apply `minimize_cost` with a scalar objective, or inspect the front and pick by hand.

Catalog Cartesian product can blow up. When several subsystems return multi-valued antichains, the inner h of the `System` takes their Cartesian product. For typical models (one or two catalog subsystems among many algebraic ones), this is not a bottleneck; but if many subsystems are catalog-driven, the inner antichain may grow large between Min prunings.

Numerical precision in feedback loops. `Antichain.eq` compares points exactly. For contractive floating-point feedback, the iteration may oscillate by an ulp near the fixed point and consume many iterations before converging. The divergence cap and `max_iter` guard against runaway, but choosing `max_iter` generously (200 to 500) is wise for nontrivial models.

17 Limitations and Future Work

Version 0.1.0 covers the algorithmic core: posets, antichains, six primitive DP types, three composition operators, the Kleene solver, and the two builders. The following are intentional omissions, all tractable extensions:

- **MCDPL surface syntax**. The paper's `mcdp { ... }` text format is not parsed. The `MCDP` builder gives the same shape in Python.
- **Non-finitely-representable antichains**. The library works on finite, exactly-represented antichains. For relations whose Pareto front is continuous, the only support is the bracket pattern of `UncertainDP`. Approximation kernels (Sec. VII of Censi 2015) are not yet implemented.
- **Visualisation of the design graph**. Example 5 visualises the Kleene ascent over $\mathbb{N} \times \mathbb{N}$, but there is no general renderer for the operator tree or for the subsystem network of a `System`.
- **Symbolic/automatic differentiation of h.** All relations are evaluated numerically.
- **Distributed or parallel solving**. The Kleene iteration is sequential.

Contributions, particularly on these items, are welcome.

18 API Reference Summary

The public API exported by `import codesign` is summarised below.

Posets. `Reals`, `Naturals`, `Ports`, `Discrete`, abstract base `Poset`.

Antichains. `Antichain`.

Design problems. `DesignProblem` (abstract base), `AlgebraicDP`, `FunctionDP`, `CatalogDP`, `CatalogEntry`, `ConstraintDP`, `ODE_DP`, `UncertainDP`.

Composition. `Series`, `Parallel`, `Loop` (classes); `series`, `par`, `loop` (function aliases).

Primitives. `adder`, `multiplier`, `scale`, `constant`, `identity`.

Solver. `solve`, `kleene_loop`, `minimize_cost`, `SolveResult`, `TraceEntry`. Warm-start via the `start_from=` argument on `solve`.

Builders. `MCDP`, `System`.

Class-based modules. `Module` (declarative `DesignProblem` base class).

Constraint DSL. `Port`, `Expr`, `ModuleHandle` (returned by `System.add`), and the helper functions `sqrt`, `exp`, `log` for use inside constraint expressions.

Uncertainty. Set-based: `UncertaintySet` (abstract), `Box`, `Ellipsoid`, `Disk`, `Circle`. Stochastic: `Stochastic`, plus copulas `Independence` and `GaussianCopula`. Result: `UncertaintyResult`. Entry point: `solve_with_uncertainty` (the same machinery is also reachable through `solve(dp, f, uncertainty=[...])`).

Online learning. `OptimisticEvaluator` (abstract); `MonotonicityEvaluator`, `LipschitzEvaluator`, `LinearParametricEvaluator`. Result: `OnlineResult`. Entry point: `solve_online`.

Visualisation. Imported as `from codesign import viz`. Functions: `plot_antichain`, `plot_convergence`, `plot_uncertainty`, `to_dot`.

Module-level. `__version__` (the string `"0.1.0"`).

References

The framework references [1]–[4] underpin the theory and software. Entries [5]–[11] are the parameter-calibration sources for the monoclonal-antibody bioprocess examples (15 and 16); entries [12]–[22] are the calibration sources for the full-vehicle co-design example (17); entries [23]–[35] underpin the temporal, vector-state, and online co-design layers (sections 14–15, examples 18–22) and the realistic problem domains surveyed there. All calibration values are illustrative, drawn from the published ranges in these sources, and are not OEM- or product-specific.

Framework and theory

[1] Andrea Censi. *A Mathematical Theory of Co-Design.* arXiv:1512.08055, 2015. https://arxiv.org/abs/1512.08055

[2] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order.* Cambridge University Press, 2nd ed., 2002.

[3] Saud Alharbi, Munther A. Dahleh, and Gioele Zardini. *Compositional Online Learning for Co-Design.* arXiv:2604.22624, 2026. https://arxiv.org/abs/2604.22624

[4] The MCDPL software distribution. https://co-design.science/software/

Bioprocess examples (15, 16)

[5] S.K. Yoon, S.H. Kim, and G.M. Lee. Effect of low culture temperature on specific productivity and transcription level of anti-4-1BB antibody in recombinant Chinese hamster ovary (CHO) cells. *Biotechnology Progress* 19:1383–1386, 2003.

[6] S.N. Sou, C. Sellick, K. Lee, et al. How does mild hypothermia affect monoclonal antibody glycosylation? *Biotechnology and Bioengineering* 112(6):1165–1176, 2015.

[7] E. Trummer, K. Fauland, S. Seidinger, et al. Process parameter shifting: Part I. Effect of DOT, pH, and temperature on the performance of Epo-Fc expressing CHO cells cultivated in controlled batch bioreactors. *Biotechnology and Bioengineering* 94(6):1033–1044, 2006.

[8] S.F. Khattak, Z. Xing, B. Kenty, I. Koyrakh, and Z.J. Li. Feed development for a fed-batch CHO production process by semi-steady-state analysis. *Biotechnology Progress* 26(3):797–804, 2010.

[9] M. Gagnon, G. Hiller, Y.-T. Luan, A. Kittredge, J. DeFelice, and D. Drapeau. High-end pH-controlled delivery of glucose (HIPDOG) effectively suppresses lactate accumulation in CHO fed-batch cultures. *Biotechnology and Bioengineering* 108(6):1328–1337, 2011.

[10] D. Reinhart, L. Damjanovic, C. Kaisermayer, W. Sommeregger, A. Gili, B. Gasselhuber, et al. Bioprocessing of recombinant CHO-K1, CHO-DG44, and CHO-S: CHO expression hosts favor either mAb production or biomass synthesis. *Biotechnology Journal* 14(3):e1700686, 2019.

[11] C. Lao and D. Toth. Effects of ammonium and lactate on growth and metabolism of a recombinant CHO cell culture. *Biotechnology Progress* 13:688–691, 1997.

Automotive co-design example (17)

[12] G. Genta and L. Morello. *The Automotive Chassis, Vol. 1: Components Design*. Springer, 2009. (Suspension stiffness, damper characteristics, brake sizing, tire load ratings.)

[13] Robert Bosch GmbH. *Bosch Automotive Handbook*, 10th ed., 2018. (Engine specific power, exhaust aftertreatment, electrical loads.)

[14] W. W. Pulkrabek. *Engineering Fundamentals of the Internal Combustion Engine*, 2nd ed. Pearson, 2003. (Thermal efficiency, heat rejection, fuel flow rates.)

[15] J. B. Heywood. *Internal Combustion Engine Fundamentals*, 2nd ed. McGraw-Hill, 2018. (Combustion thermodynamics, BSFC ranges, durability.)

[16] P. Hofmann. *Hybridfahrzeuge*, 2nd ed. Springer, 2014. (Power-split topologies, sizing rules, efficiency models.)

[17] H. Naunheimer et al. *Automotive Transmissions*, 2nd ed. Springer, 2011. (Gearbox losses, weights, costs.)

[18] International Energy Agency. *Global EV Outlook*, 2023. (Battery pack cost and pack-level energy-density trends, 2020–2030.)

[19] F. Nemry et al. *Environmental Improvement of Passenger Cars*. JRC Scientific and Technical Reports, 2008. (CO_2 emission factors, fuel densities, well-to-wheel.)

[20] U.S. Environmental Protection Agency. *Automotive Trends Report*, 2024. (Fleet-average fuel economy, weight-class data, drag-coefficient data.)

[21] J. Larminie and J. Lowry. *Electric Vehicle Technology Explained*, 2nd ed. Wiley, 2012. (Motor efficiency maps, inverter losses, charging-system architecture.)

[22] P. Mock et al. *European Vehicle Market Statistics Pocketbook 2023/24*. International Council on Clean Transportation, 2023. (Mass-versus-mission scatter data for calibration.)

Temporal, vector-state, and online co-design (18–22, sections 14–15)

[23] M.-P. Neumann, R. Habermacher, G. Fieni, A. Cerofolini, G. Zardini, and C. H. Onder. Hierarchical Co-Design for Multi-Race Strategy Optimization in Formula 1. In *29th IEEE Intelligent Transportation Systems Conference (ITSC)*, 2026 (in press). ETH Research Collection https://doi.org/10.3929/ethz-c-000784888. (The precompute-then-DP structure and the multi-component seasonal state vector.)

[24] D. R. Herber and J. T. Allison. Nested and Simultaneous Solution Strategies for General Combined Plant and Control Design Problems. *Journal of Mechanical Design* 141(1):011402, 2018. (The nested vs simultaneous CCD paradigms.)

[25] M. Garcia-Sanz. Control Co-Design: An engineering game changer. *Advanced Control for Applications* 1(1):e18, 2019.

[26] Self-reconfiguring modular robots (overview). See e.g. *Modular reconfigurable robots: Toward on-demand multifunctional applications*, Science Robotics, 2024, and the survey *Modular Self-Configurable Robots—The State of the Art*, Actuators 12(9):361, 2023.

[27] Z. Liu, Q. Lu, J. Luo, and Z. Wang. Terrain-aware morphology searching algorithm for self-reconfigurable modular robot in dynamic environment. *Applied Soft Computing* 186:114182, 2026.

[28] X. An, Q. Jia, G. Chen, Y. Liu, and H. Liu. Morphological Design and Performance Analysis of a Self-reconfigurable Modular Robot for Multi-task Space Applications. *Journal of Mechanical Engineering* 61(21):152–167, 2025.

[29] Modular Self-Configurable Robots—The State of the Art. *Actuators* 12(9):361, 2023. (Floor-cleaning tiling robots hTrihex/hTetro and reconfiguration for coverage.)

[30] M. C. Hübl and C. P. Goodrich. Simultaneous optimization of assembly time and yield in programmable self-assembly. *The Journal of Chemical Physics* 164(8):084904, 2026.

[31] M. C. Hübl and C. P. Goodrich. Accessing Semi-Addressable Self-Assembly with Efficient Structure Enumeration. *Physical Review Letters* 134(5):058204, 2025.

[32] Optimization of Non-Equilibrium Self-Assembly Protocols Using Markov State Models. arXiv:2210.05749. (Staged assembly-protocol optimisation.)

[33] Multi-objective protein design (representative works): *Searching for the Pareto frontier in multi-objective protein design*, Biophysical Reviews 9:311–324, 2017; MosPro, *Pareto-optimal sampling for multi-objective protein sequence design*, 2025; ST-PARM and MAProt, bioRxiv, 2026; EVOLVEpro (closed-loop directed evolution).

[34] J. Sun et al. Robust Control Co-Design with Receding-Horizon MPC. arXiv:2104.02025. (The online-feedback co-design pattern.)

[35] D. J. Docimo, Z. Kang, K. A. James, and A. G. Alleyne. Plant and Controller Optimization for Power and Energy Systems with Model Predictive Control. *Journal of Dynamic Systems, Measurement, and Control*, 2021.